

```
===== 文件: app.js =====
const auth = require('./utils/auth');
const { post } = require('./utils/request');
const { captureNfcClaimIntent } = require('./utils/nfc-claim-intent');
const shareInvite = require('./utils/share-invite');

const AUTH_FIELDS = ['token', 'userId', 'openid', 'isNewUser', 'hasPhone', 'agentId'];

function loginWithWechat() {
  return new Promise((resolve, reject) => {
    wx.login({
      success: (result) => {
        if (!result || !result.code) {
          reject(new Error('微信登录失败，请重试'));
          return;
        }
        resolve(result.code);
      },
      fail: () => reject(new Error('微信登录失败，请重试'))
    });
    }).then((code) => post('/wechat/login', { code }, { anonymous: true }));
  }
}

function saveShareInviteContext(options) {
  const context = shareInvite.parseEntryOptions(options);
  if (context) shareInvite.savePending(context);
}

App({
  globalData: {
    version: '1.0.0-mvp',
    authReady: null,
    token: null,
    userId: null,
    openid: null,
    isNewUser: null,
    hasPhone: null,
    agentId: null,
    welcomeCompleted: false,
    launchPath: 'pages/home/home'
  },

  onLaunch(options) {
    captureNfcClaimIntent(options);
    saveShareInviteContext(options);
    this.globalData.launchPath = options && options.path
      ? options.path
      : 'pages/home/home';
    this.globalData.authReady = this.ensureLogin()
      .then((session) => {
        this.redirectUnboundToWelcome(this.globalData.launchPath);
        return session;
      })
      .catch(() => {
        this.applySession(null);
        return null;
      });
  },

  onShow(options) {
```

```

captureNfcClaimIntent(options);
saveShareInviteContext(options);
const session = auth.getSession();
if (session && auth.isExpired()) {
  this.clearLoginState();
  return;
}
if (session && auth.isExpiringSoon()) {
  this.silentLogin().catch(() => null);
}
const pages = typeof getCurrentPages === 'function' ? getCurrentPages() : [];
const currentRoute = pages.length ? pages[pages.length - 1].route : this.globalData.launchPath;
this.redirectUnboundToWelcome(currentRoute);
},

redirectUnboundToWelcome(route) {
  if (route === 'pages/welcome/welcome' || route === '/pages/welcome/welcome') {
    this._welcomeRedirecting = false;
    return;
  }
  if (this.globalData.welcomeCompleted === true) return;
  if (this._welcomeRedirecting === true) return;
  if (!auth.getSession()) return;
  if (this.globalData.hasPhone !== true) {
    this._welcomeRedirecting = true;
    const redirect = () => wx.reLaunch({ url: '/pages/welcome/welcome' });
    if (typeof wx.nextTick === 'function') wx.nextTick(redirect);
    else setTimeout(redirect, 0);
  }
},

applySession(session) {
  AUTH_FIELDS.forEach((field) => {
    this.globalData[field] = session ? session[field] : null;
  });
},

silentLogin() {
  if (this._loginPromise) return this._loginPromise;
  this._loginPromise = loginWithWechat()
    .then((loginData) => {
      const session = auth.saveSession(loginData);
      this.applySession(session);
      return session;
    });
  this._loginPromise.then(
    () => { this._loginPromise = null; },
    () => { this._loginPromise = null; }
  );
  return this._loginPromise;
},

ensureLogin() {
  const session = auth.getSession();
  if (session && !auth.isExpired() && !auth.isExpiringSoon()) {
    this.applySession(session);
    return Promise.resolve(session);
  }
  return this.silentLogin();
},

```

```

clearLoginState() {
  auth.clearSession();
  this.applySession(null);
}
});
===== 文件: config/age-ranges.js =====
// 年龄区间本地兜底：仅在字典接口拉取失败时使用，正式数据以智控台字典管理（EGG_AGE_RANGE）为准
const FALLBACK_AGE_RANGES = Object.freeze([
  Object.freeze({ value: 'AGE_0_14', label: '14 周岁及以下' }),
  Object.freeze({ value: 'AGE_15_35', label: '15-35 周岁' }),
  Object.freeze({ value: 'AGE_36_60', label: '36-60 周岁' }),
  Object.freeze({ value: 'AGE_61_PLUS', label: '60 周岁以上' })
]);

module.exports = { FALLBACK_AGE_RANGES };
===== 文件: config/api.js =====
// const API_BASE_URL = 'http://192.168.3.75:8002/xiaozhi';

const API_BASE_URL = 'https://api.eggbabe.com/xiaozhi';
const OSS_SCENE_BASE = 'https://oss.eggbabe.com/scenes/pre-hatch';

module.exports = { API_BASE_URL, OSS_SCENE_BASE };
===== 文件: config/feedback-types.js =====
// 诉求类型本地兜底：仅在字典接口拉取失败时使用，正式数据以智控台字典管理（EGG_FEEDBACK_TYPE）为准
const FALLBACK_FEEDBACK_TYPES = Object.freeze([
  Object.freeze({ value: 'AI_CONTENT_VIOLATION', label: 'AI 虚拟宠物对话违规（涉政、色情、暴力、轻生、暧昧诱导）' }),
  Object.freeze({ value: 'AI_MISLEADING_ADVICE', label: 'AI 误导或给出不实医疗、理财建议' }),
  Object.freeze({ value: 'MINOR_USE', label: '未成年人使用相关问题（时长、模式、内容限制）' }),
  Object.freeze({ value: 'SENIOR_SUPPORT', label: '老年人操作、防诈骗或使用指引咨询' }),
  Object.freeze({ value: 'PRIVACY', label: '账号或聊天记录隐私保护问题' }),
  Object.freeze({ value: 'CRISIS_INTERVENTION', label: '极端情绪干预失效（自杀、抑郁、家暴未安抚）' }),
  Object.freeze({ value: 'FUNCTION_FAILURE', label: '功能故障或无法打开宠物对话' }),
  Object.freeze({ value: 'REMINDER_OR_AI_LABEL', label: '时长提醒或 AI 标识缺失' }),
  Object.freeze({ value: 'OTHER', label: '其他申诉、意见建议' })
]);

module.exports = { FALLBACK_FEEDBACK_TYPES };
===== 文件: config/pre-hatch-assets.js =====
// 破壳前场景素材集中配置。整幅背景/窝垫/蛋/窗景均托管 OSS，本地只保留蛋壳 overlay 与入口图标。
const { OSS_SCENE_BASE } = require('./api');
const FULL_SCENE_ROOT = `${OSS_SCENE_BASE}/incubation-room/season-weather-full-scenes`;
const NEST_SCENE_ROOT = `${OSS_SCENE_BASE}/nest/season-weather`;
const EGG_SCENE_ROOT = `${OSS_SCENE_BASE}/egg/season-weather`;
const WINDOW_SCENE_ROOT = `${OSS_SCENE_BASE}/window/window-weather`;

const SCENE_TABLE = [
  ['spring_clear_day', 'spring', 'sunny', 'day', 'midday'],
  ['spring_clear_sunset', 'spring', 'sunny', 'sunset', 'sunset'],
  ['spring_clear_night', 'spring', 'sunny', 'night', 'night'],
  ['spring_cloudy_day', 'spring', 'cloudy', 'day', 'midday'],
  ['spring_cloudy_sunset', 'spring', 'cloudy', 'sunset', 'sunset'],
  ['spring_cloudy_night', 'spring', 'cloudy', 'night', 'night'],
  ['spring_rain_day', 'spring', 'rain', 'day', 'midday'],
  ['spring_rain_sunset', 'spring', 'rain', 'sunset', 'sunset'],
  ['spring_rain_night', 'spring', 'rain', 'night', 'night'],
  ['summer_clear_day', 'summer', 'sunny', 'day', 'midday'],
  ['summer_clear_sunset', 'summer', 'sunny', 'sunset', 'sunset'],
  ['summer_clear_night', 'summer', 'sunny', 'night', 'night'],

```

```

['summer_cloudy_day', 'summer', 'cloudy', 'day', 'midday'],
['summer_cloudy_sunset', 'summer', 'cloudy', 'sunset', 'sunset'],
['summer_cloudy_night', 'summer', 'cloudy', 'night', 'night'],
['summer_storm_day', 'summer', 'storm', 'day', 'midday'],
['summer_storm_sunset', 'summer', 'storm', 'sunset', 'sunset'],
['summer_storm_night', 'summer', 'storm', 'night', 'night'],
['autumn_clear_day', 'autumn', 'sunny', 'day', 'midday'],
['autumn_clear_sunset', 'autumn', 'sunny', 'sunset', 'sunset'],
['autumn_clear_night', 'autumn', 'sunny', 'night', 'night'],
['autumn_rain_day', 'autumn', 'rain', 'day', 'midday'],
['autumn_rain_sunset', 'autumn', 'rain', 'sunset', 'sunset'],
['autumn_rain_night', 'autumn', 'rain', 'night', 'night'],
['winter_clear_day', 'winter', 'sunny', 'day', 'midday'],
['winter_clear_sunset', 'winter', 'sunny', 'sunset', 'sunset'],
['winter_clear_night', 'winter', 'sunny', 'night', 'night'],
['winter_cloudy_day', 'winter', 'cloudy', 'day', 'midday'],
['winter_cloudy_sunset', 'winter', 'cloudy', 'sunset', 'sunset'],
['winter_cloudy_night', 'winter', 'cloudy', 'night', 'night'],
['winter_snow_day', 'winter', 'snow', 'day', 'midday'],
['winter_snow_sunset', 'winter', 'snow', 'sunset', 'sunset'],
['winter_snow_night', 'winter', 'snow', 'night', 'night'],
['winter_post_snow_day', 'winter', 'postSnow', 'day', 'midday'],
['winter_post_snow_sunset', 'winter', 'postSnow', 'sunset', 'sunset'],
['winter_post_snow_night', 'winter', 'postSnow', 'night', 'night']
];

```

```

const SCENE_OPTIONS = SCENE_TABLE.map([(key, season, weather, period, lightPhase)] => ({
  key, season, weather, period, lightPhase,
  className: `season-${season} weather-${weather} period-${period} light-${lightPhase}`,
  background: `${FULL_SCENE_ROOT}/${key}.webp`,
  nest: `${NEST_SCENE_ROOT}/${key}_nest_pad.webp`,
  egg: `${EGG_SCENE_ROOT}/${key}_egg_right45.webp`
}));

```

```

const WINDOW_WEATHER = {
  clearDay: `${WINDOW_SCENE_ROOT}/w_01_clear_day.webp`,
  clearSunset: `${WINDOW_SCENE_ROOT}/w_02_clear_sunset.webp`,
  clearNight: `${WINDOW_SCENE_ROOT}/w_03_clear_night.webp`,
  cloudyDay: `${WINDOW_SCENE_ROOT}/w_04_cloudy_day.webp`,
  cloudyNight: `${WINDOW_SCENE_ROOT}/w_05_cloudy_night.webp`,
  snowDay: `${WINDOW_SCENE_ROOT}/w_06_snow_day.webp`,
  snowNight: `${WINDOW_SCENE_ROOT}/w_07_snow_night.webp`,
  // 16 个环境键的精确窗景；优先于共享图。
  bySceneKey: {
    spring_cloudy_sunset: `${WINDOW_SCENE_ROOT}/window_spring_cloudy_sunset_v01.webp`,
    spring_rain_day: `${WINDOW_SCENE_ROOT}/window_spring_rain_day_v01.webp`,
    spring_rain_sunset: `${WINDOW_SCENE_ROOT}/window_spring_rain_sunset_v01.webp`,
    spring_rain_night: `${WINDOW_SCENE_ROOT}/window_spring_rain_night_v01.webp`,
    summer_cloudy_sunset: `${WINDOW_SCENE_ROOT}/window_summer_cloudy_sunset_v01.webp`,
    summer_storm_day: `${WINDOW_SCENE_ROOT}/window_summer_storm_day_v01.webp`,
    summer_storm_sunset: `${WINDOW_SCENE_ROOT}/window_summer_storm_sunset_v01.webp`,
    summer_storm_night: `${WINDOW_SCENE_ROOT}/window_summer_storm_night_v01.webp`,
    autumn_rain_day: `${WINDOW_SCENE_ROOT}/window_autumn_rain_day_v01.webp`,
    autumn_rain_sunset: `${WINDOW_SCENE_ROOT}/window_autumn_rain_sunset_v01.webp`,
    autumn_rain_night: `${WINDOW_SCENE_ROOT}/window_autumn_rain_night_v01.webp`,
    winter_cloudy_sunset: `${WINDOW_SCENE_ROOT}/window_winter_cloudy_sunset_v01.webp`,
    winter_snow_sunset: `${WINDOW_SCENE_ROOT}/window_winter_snow_sunset_v01.webp`,
    winter_post_snow_day: `${WINDOW_SCENE_ROOT}/window_winter_post_snow_day_v01.webp`,
    winter_post_snow_sunset: `${WINDOW_SCENE_ROOT}/window_winter_post_snow_sunset_v01.webp`,
    winter_post_snow_night: `${WINDOW_SCENE_ROOT}/window_winter_post_snow_night_v01.webp`
  }
};

```

```

    }
  };

const EGG_ASSET_ROOT = '/assets/scenes/egg';

module.exports = {
  SCENE_OPTIONS,
  WINDOW_WEATHER,
  // 本地打包资源统一用 PNG：微信上传管线对 .webp 资源可能漏打包导致生产不显示，PNG 稳定
  EGG_DEPTH_OVERLAY: `${EGG_ASSET_ROOT}/egg_shell_depth_overlay_512_v01.png`,
  EGG_SPECULAR_OVERLAY: `${EGG_ASSET_ROOT}/egg_shell_specular_overlay_512_v01.png`,
  INTERACTION_ICONS: {
    wish: '/assets/ui/3d-actions/ui_3d_wishing_fountain_two_tier_simple_256_v04.png',
    learn: '/assets/ui/3d-actions/ui_3d_early_learning_picture_book_simple_256_v03.png',
    draw: '/assets/ui/3d-actions/ui_3d_drawing_palette_256_v02.png'
  }
};

===== 文件: config/wish-questions.js =====
/**
 * 许愿池题目静态配置（PRD 5.2.1）
 * 共 7 道单选题，每道题选项带 emoji。
 * 题目为产品文案，放在前端本地，不建后端表。
 * 用 id 唯一标识题目，便于匹配已答记录。
 */
module.exports = [
  {
    id: 'world-color',
    title: '你希望蛋宝宝破壳后，第一眼看到的"世界"是什么颜色？',
    options: [
      { emoji: '🌿', text: '森林绿' },
      { emoji: '🌊', text: '海洋蓝' },
      { emoji: '🌸', text: '樱花粉' },
      { emoji: '🌅', text: '夕阳橙' },
      { emoji: '🌌', text: '星空紫' }
    ]
  },
  {
    id: 'sleep-type',
    title: '你希望蛋宝宝是早睡早起型，还是陪你熬夜型？',
    options: [
      { emoji: '🐦', text: '早鸟型' },
      { emoji: '🐱', text: '夜猫型' },
      { emoji: '🌙', text: '跟随型' }
    ]
  },
  {
    id: 'gender',
    title: '你希望蛋宝宝破壳后，性别是？',
    options: [
      { emoji: '👦', text: '小男孩' },
      { emoji: '👧', text: '小女孩' },
      { emoji: '🌈', text: '让蛋宝宝自己选' }
    ]
  },
  {
    id: 'life-type',
    title: '你更希望蛋宝宝像哪种小生命？',
    options: [
      { emoji: '☀️', text: '小太阳' },
      { emoji: '🌙', text: '小月亮' },

```

```

    { emoji: '🌬️', text: '小风' },
    { emoji: '🪨', text: '小石头' }
  ],
},
{
  id: 'animal-soul',
  title: '你希望蛋宝宝是哪种小动物的灵魂? ',
  options: [
    { emoji: '🐶', text: '小狗型' },
    { emoji: '🐱', text: '小猫型' },
    { emoji: '🐰', text: '小兔型' },
    { emoji: '🌳', text: '小树懒型' }
  ]
},
{
  id: 'world-taste',
  title: '你希望蛋宝宝睁开眼睛时，这个世界是什么味道的? ',
  options: [
    { emoji: '🌿', text: '雨后青草味' },
    { emoji: '🍞', text: '烤面包味' },
    { emoji: '🌸', text: '晒过的被子味' },
    { emoji: '🍊', text: '橘子汽水味' }
  ]
},
{
  id: 'inner-weather',
  title: '如果蛋宝宝身体里住着一种天气，你希望是? ',
  options: [
    { emoji: '☀️', text: '晴天' },
    { emoji: '🌧️', text: '小雨' },
    { emoji: '☁️', text: '多云' },
    { emoji: '🌈', text: '彩虹' }
  ]
}
];
===== 文件: services/analytics.js =====
/**
 * analytics stub — 轻量空实现
 * life-scene 等页面调用 analytics.track() 时静默跳过，不报错。
 */
function track() {
  return { ok: true };
}

function flush() {
  return Promise.resolve({ ok: true, count: 0 });
}

module.exports = { track, flush };
===== 文件: services/time-service.js =====
/**
 * time-service 轻量实现
 * 直接使用本地时间，不含服务端时间同步逻辑。
 */
function now() {
  return Date.now();
}

function beijingDateKey(timestamp) {
  const date = new Date((timestamp === undefined ? now() : timestamp) + 8 * 60 * 60 * 1000);

```

```

    return date.toISOString().slice(0, 10);
  }

function formatBeijingDate(timestamp) {
  const key = beijingDateKey(timestamp);
  const parts = key.split('-').map(Number);
  return parts[0] + '年' + parts[1] + '月' + parts[2] + '日';
}

function sync() {
  return Promise.resolve({ ok: true, now: now() });
}

function isAuthoritative() {
  return true;
}

function requireAuthoritative() {
  return { ok: true, now: now(), authoritative: true };
}

module.exports = { now, beijingDateKey, formatBeijingDate, sync, isAuthoritative, requireAuthoritative }
;
===== 文件: utils/age-range-api.js =====
const { get, put } = require('./request');

// 年龄区间字典类型编码，与后端 sys_dict_type 对应
const AGE_RANGE_DICT = 'EGG_AGE_RANGE';

// 从后端字典拉取年龄区间选项，SysDictDataItem: [{ name, key }] -> [{ label, value }]
function listAgeRanges() {
  return get('/admin/dict/data/type/${AGE_RANGE_DICT}`).then((items) => {
    if (!Array.isArray(items)) return [];
    return items
      .filter((item) => item && item.key && item.name)
      .map((item) => ({ label: item.name, value: item.key }));
  });
}

// 读取用户资料（含 ageRange 字段）
function getProfile() {
  return get('/wechat/profile');
}

// 保存年龄区间，复用资料更新接口的部分字段更新能力
function saveAgeRange(ageRange) {
  return put('/wechat/profile', { ageRange });
}

module.exports = { listAgeRanges, getProfile, saveAgeRange };
===== 文件: utils/audio.js =====
/**
 * utils/audio.js
 * -----
 * AudioManager — playback-only variant for the egg-miniprogram chat.
 *
 * Receives Opus frames from the xiaozhi-server WebSocket, decodes them,
 * and plays them gaplessly through wx.createWebAudioContext().
 *
 * Lifecycle:

```

```
* const audio = new AudioManager({ onPlayEnd, onError });
* await audio.ready();
* audio.appendOpusFrame(frame);
* audio.stopPlayback();
* audio.destroy();
*
* Notes:
* - WebAudioContext requires base library >= 2.19.0.
* - Recording/encoding code is intentionally omitted; this file only handles
*   inbound audio playback.
* -----
*/

const OpusDecoder = require('../libs/opus/opus-decoder');

const SAMPLE_RATE = 24000;
const CHANNELS = 1;
const FRAME_DURATION_MS = 60;
const FRAME_SAMPLES = SAMPLE_RATE * FRAME_DURATION_MS / 1000; // 1440

class AudioManager {
  /**
   * @param {Object} options
   * @param {} => void} [options.onPlayEnd] Fires when playback queue drains.
   * @param {(err: Error, scope: string) => void} [options.onError]
   */
  constructor(options) {
    this.options = options || {};

    this.decoder = new OpusDecoder({
      sampleRate: SAMPLE_RATE,
      channels: CHANNELS,
      frameDuration: FRAME_DURATION_MS,
    });

    this._readyPromise = this.decoder.ready()
      .then(() => {
        this._codecMode = this.decoder.mode;
        if (this._codecMode === 'stub') {
          this._emitError(new Error('Opus 解码器回退到 stub, 真 Opus 不可用'), 'codec');
        }
        return this;
      });

    // Playback state
    this._audioCtx = null;
    this._playQueue = [];
    this._isPlaying = false;
    this._nextStartTime = 0;
    this._activeSources = new Set();

    this._destroyed = false;
    this._codecMode = null;
  }

  /** Resolves when the decoder runtime is loaded. */
  ready() {
    return this._readyPromise;
  }
}
```



```
// -----  
// Playback  
// -----  
  
/**  
 * Decode and enqueue an array of opus frames.  
 */  
playOpusFrames(frames) {  
  if (!frames || !frames.length) return;  
  for (let i = 0; i < frames.length; i++) {  
    this.appendOpusFrame(frames[i]);  
  }  
}  
  
/**  
 * Decode one frame and append it to the play queue.  
 */  
appendOpusFrame(frame) {  
  if (this._destroyed || !frame) return;  
  let pcm;  
  try {  
    pcm = this.decoder.decode(frame);  
  } catch (e) {  
    this._emitError(e, 'decode');  
    return;  
  }  
  if (!pcm || pcm.length === 0) return;  
  
  this._playQueue.push(pcm);  
  this._scheduleQueued();  
}  
  
stopPlayback() {  
  this._playQueue.length = 0;  
  this._stopActiveSources();  
  this._isPlaying = false;  
  this._nextStartTime = 0;  
}  
  
/**  
 * Recreate the WebAudioContext so the next playback starts cleanly.  
 */  
resetAudioContext() {  
  if (this._destroyed) return;  
  this._playQueue.length = 0;  
  this._stopActiveSources();  
  this._isPlaying = false;  
  this._nextStartTime = 0;  
  
  const oldCtx = this._audioCtx;  
  this._audioCtx = this._createAudioContext();  
  
  if (oldCtx && typeof oldCtx.close === 'function') {  
    try { oldCtx.close(); } catch (_) {}  
  }  
}  
  
_stopActiveSources() {  
  if (!this._activeSources || this._activeSources.size === 0) return;  
  try {
```

```
this._activeSources.forEach((src) => {
  try { src.stop(0); } catch (_) {}
  try { src.disconnect(); } catch (_) {}
});
} finally {
  this._activeSources.clear();
}
}

_createAudioContext() {
  if (typeof wx === 'undefined' || !wx.createWebAudioContext) {
    this._emitError(new Error('wx.createWebAudioContext requires base lib >= 2.19.0'), 'play');
    return null;
  }
  return wx.createWebAudioContext();
}

_ensureAudioContext() {
  if (this._audioCtx) return this._audioCtx;
  this._audioCtx = this._createAudioContext();
  return this._audioCtx;
}

_scheduleQueued() {
  const ctx = this._ensureAudioContext();
  if (!ctx) return;

  while (this._playQueue.length > 0) {
    const pcm = this._playQueue.shift();
    const buffer = ctx.createBuffer(CHANNELS, pcm.length, SAMPLE_RATE);
    const channelData = buffer.getChannelData(0);
    for (let i = 0; i < pcm.length; i++) {
      channelData[i] = pcm[i] / 32768;
    }

    const source = ctx.createBufferSource();
    source.buffer = buffer;
    source.connect(ctx.destination);

    const now = ctx.currentTime;
    const startAt = Math.max(now, this._nextStartTime);
    try {
      source.start(startAt);
    } catch (e) {
      this._emitError(e, 'play');
      continue;
    }
    this._nextStartTime = startAt + buffer.duration;
    this._isPlaying = true;
    this._activeSources.add(source);

    source.onended = () => {
      this._activeSources.delete(source);
      try { source.disconnect(); } catch (_) {}
      if (this._activeSources.size === 0 && this._playQueue.length === 0) {
        this._isPlaying = false;
        this._nextStartTime = 0;
        if (this.options.onPlayEnd) this.options.onPlayEnd();
      }
    };
  }
};
```

```

    }
  }

  // -----
  // Lifecycle
  // -----

  destroy() {
    if (this._destroyed) return;
    this._destroyed = true;
    try { this.stopPlayback(); } catch (_) {}
    if (this._audioCtx && typeof this._audioCtx.close === 'function') {
      try { this._audioCtx.close(); } catch (_) {}
    }
    this._audioCtx = null;
    if (this.decoder) this.decoder.destroy();
    this.decoder = null;
  }

  // -----
  // Helpers
  // -----

  _emitError(err, scope) {
    if (this.options && this.options.onError) {
      try { this.options.onError(err, scope); } catch (_) {}
    } else {
      console.error('[AudioManager:' + scope + ']', err);
    }
  }
}

module.exports = AudioManager;
module.exports.AudioManager = AudioManager;
module.exports.SAMPLE_RATE = SAMPLE_RATE;
module.exports.FRAME_SAMPLES = FRAME_SAMPLES;
module.exports.FRAME_DURATION_MS = FRAME_DURATION_MS;
===== 文件: utils/auth.js =====
const PREFIX = 'eggbaby_auth_';
const KEYS = {
  token: `${PREFIX}token`, userId: `${PREFIX}user_id`, openid: `${PREFIX}openid`,
  isNewUser: `${PREFIX}is_new_user`, hasPhone: `${PREFIX}has_phone`,
  agentId: `${PREFIX}agent_id`, issuedAt: `${PREFIX}issued_at`, expire: `${PREFIX}expire`
};
const REQUIRED = ['token', 'userId', 'openid', 'expire'];

function saveSession(session, issuedAt) {
  if (!session || REQUIRED.some((key) => session[key] === undefined || session[key] === null || session[key] === '')) {
    throw new Error('登录响应缺少必要字段');
  }
  const normalized = {
    token: session.token, userId: session.userId, openid: session.openid,
    isNewUser: !!session.isNewUser, hasPhone: !!session.hasPhone,
    agentId: session.agentId || null, issuedAt: issuedAt || Date.now(), expire: Number(session.expire)
  };
  if (!Number.isFinite(normalized.expire) || normalized.expire <= 0) throw new Error('登录响应缺少必要字段');
  Object.keys(KEYS).forEach((key) => wx.setStorageSync(KEYS[key], normalized[key]));
  return normalized;
}

```

```

}

function getSession() {
  const session = {};
  Object.keys(KEYS).forEach((key) => { session[key] = wx.getStorageSync(KEYS[key]); });
  if (REQUIRED.some((key) => session[key] === undefined || session[key] === null || session[key] === ""))
    return null;
  return session;
}

function expiresAt(session) { return session.issuedAt + session.expire * 1000; }
function isExpired(now) { const s = getSession(); return !s || (now || Date.now()) >= expiresAt(s); }
function isExpiringSoon(now, bufferSeconds) {
  const s = getSession();
  return !s || (now || Date.now()) + (bufferSeconds === undefined ? 300 : bufferSeconds) * 1000 >= expiresAt(s);
}
function isValidSession(now) { return !isExpired(now); }
function clearSession() { Object.values(KEYS).forEach((key) => wx.removeStorageSync(key)); }
function markPhoneBound() {
  const session = getSession();
  if (!session) return null;
  return saveSession({ ...session, hasPhone: true }, session.issuedAt);
}

module.exports = { saveSession, getSession, clearSession, markPhoneBound, isValidSession, isExpired, isExpiringSoon };
===== 文件: utils/canvas-2d.js =====
function createLayer(page, selector) {
  return new Promise(resolve => {
    wx.createSelectorQuery().in(page).select(selector).fields({ node: true, size: true, rect: true }).exec(result => {
      const target = result[0];
      if (!target || !target.node || !target.width || !target.height) {
        resolve(null);
        return;
      }
      const windowInfo = wx.getWindowInfo ? wx.getWindowInfo() : wx.getSystemInfoSync();
      const pixelRatio = Number(windowInfo.pixelRatio || 1);
      const canvas = target.node;
      const context = canvas.getContext('2d');
      canvas.width = Math.round(target.width * pixelRatio);
      canvas.height = Math.round(target.height * pixelRatio);
      context.scale(pixelRatio, pixelRatio);
      resolve({
        canvas,
        context,
        width: target.width,
        height: target.height,
        left: target.left || 0,
        top: target.top || 0
      });
    });
  });
}

function loadImage(layer, source) {
  return new Promise(resolve => {
    if (!layer || !layer.canvas || !layer.canvas.createImage) {
      resolve(null);
    }
  });
}

```

```

    return;
  }
  const image = layer.canvas.createImage();
  image.onload = () => resolve(image);
  image.onerror = () => resolve(null);
  image.src = source;
});
}

function exportImage(layer) {
  return new Promise(resolve => {
    if (!layer || !layer.canvas || !wx.canvasToTempFilePath) {
      resolve("");
      return;
    }
    wx.canvasToTempFilePath({
      canvas: layer.canvas,
      x: 0,
      y: 0,
      width: layer.width,
      height: layer.height,
      destWidth: layer.canvas.width,
      destHeight: layer.canvas.height,
      fileType: 'png',
      quality: 1,
      success: result => resolve(result.tempFilePath || ""),
      fail: () => resolve("")
    });
  });
}

module.exports = { createLayer, loadImage, exportImage };
===== 文件: utils/device-clock.js =====
// 设备时钟快照: 为 room-clock 组件提供 文本时间/日期 与 指针角度
// 移植自 eggbase-miniprogram 静态项目 services/device-clock.js
const WEEKDAYS = ['周日', '周一', '周二', '周三', '周四', '周五', '周六'];

function pad(value) {
  return String(value).padStart(2, '0');
}

function snapshot(input) {
  const candidate = input instanceof Date ? input : new Date(input === undefined ? Date.now() : input);
  const date = Number.isFinite(candidate.getTime()) ? candidate : new Date();
  const hours = date.getHours();
  const minutes = date.getMinutes();
  const seconds = date.getSeconds();
  const milliseconds = date.getMilliseconds();
  const preciseSeconds = seconds + milliseconds / 1000;

  return {
    timeText: `${pad(hours)}:${pad(minutes)}`,
    dateText: `${date.getMonth() + 1}月${date.getDate()}日 ${WEEKDAYS[date.getDay()]}`,
    // 时针/分针随秒连续走, 避免每秒跳一格的机械感
    hourAngle: (hours % 12) * 30 + minutes * 0.5 + preciseSeconds / 120,
    minuteAngle: minutes * 6 + preciseSeconds * 0.1,
    secondAngle: preciseSeconds * 6
  };
}

```

```

// 距下一秒边界的毫秒数，用于把 1s 定时器对齐到整秒，防止指针抖动
function millisecondsUntilNextSecond(timestamp) {
  const value = Number.isFinite(Number(timestamp)) ? Number(timestamp) : Date.now();
  const remainder = ((value % 1000) + 1000) % 1000;
  return remainder === 0 ? 1000 : 1000 - remainder;
}

module.exports = {
  snapshot,
  millisecondsUntilNextSecond
};
===== 文件: utils/doodle-api.js =====
// 涂鸦图片上传与回显：图片经通用上传接口 /upload/image 传 OSS，画作出处存 hatch-action payload。
const { post, get } = require('./request');
const auth = require('./auth');
const { API_BASE_URL } = require('../config/api');

/**
 * 上传涂鸦图片到 OSS。
 * 调用通用上传接口 POST /upload/image (scene=doodle)，接口返回通用 envelope { code, data, msg }。
 * @param {string} tempFilePath 本地临时文件路径
 * @returns {Promise<string>} OSS 图片 URL
 */
function uploadDoodleImage(tempFilePath) {
  return new Promise((resolve, reject) => {
    const session = auth.getSession();
    if (!session || !session.token) {
      reject({ userMessage: '登录状态已失效' });
      return;
    }
    wx.uploadFile({
      url: `${API_BASE_URL}/upload/image`,
      filePath: tempFilePath,
      name: 'file',
      formData: { scene: 'doodle' },
      header: { Authorization: `Bearer ${session.token}` },
      success: (res) => {
        if (res.statusCode !== 200) {
          reject({ userMessage: '画作上传失败' });
          return;
        }
        try {
          const envelope = JSON.parse(res.data);
          if (envelope.code !== 0 || !envelope.data) {
            reject({ userMessage: envelope.msg || '画作上传失败' });
            return;
          }
          resolve(envelope.data);
        } catch (error) {
          reject({ userMessage: '画作上传失败' });
        }
      },
      fail: () => reject({ userMessage: '画作上传失败' })
    });
  });
}

/**
 * 查询某只宠物的最新涂鸦图片 URL。
 * 兼容 HatchActionVO 的两种字段命名 (actionType / action_type) 以及两种 payload 类型 (字符串 / 对象)。

```

```

* @param {string} petId 宠物 id
* @returns {Promise<string>} 最新涂鸦 artUrl, 无则返回空字符串
*/
async function getLatestDoodleArtUrl(petId) {
  const actions = await get(`/pet/${petId}/hatch-actions`);
  const doodles = (actions || []).filter((a) => (a.actionType || a.action_type) === 'DOODLE');
  if (!doodles.length) return "";
  // 按动作日期排序后取最新一条, 避免依赖后端返回顺序
  const sorted = doodles.slice().sort((a, b) => {
    const ta = Date.parse(a.actionDate || a.action_date || "");
    const tb = Date.parse(b.actionDate || b.action_date || "");
    return (Number.isFinite(ta) ? ta : 0) - (Number.isFinite(tb) ? tb : 0);
  });
  const latest = sorted[sorted.length - 1];
  const payload = latest.payload;
  if (!payload) return "";
  let parsed = payload;
  if (typeof payload === 'string') {
    try {
      parsed = JSON.parse(payload);
    } catch (error) {
      // 后端返回非法 JSON 时按无画作处理, 不抛原生异常破坏调用方
      return "";
    }
  }
  return parsed.artUrl || "";
}

module.exports = { uploadDoodleImage, getLatestDoodleArtUrl };
===== 文件: utils/doodle-save.js =====
const doodleApi = require('./doodle-api');
const petStore = require('./pet-store');

const DEFAULT_ERROR_MESSAGE = '画作没有保存好, 请再试一次';

function errorMessage(error) {
  return (error && (error.userMessage || error.message)) || DEFAULT_ERROR_MESSAGE;
}

async function saveArtwork({ petId, operations, exportArtwork } = {}) {
  let artUrl = "";

  try {
    if (!petId || !Array.isArray(operations)) {
      return { ok: false, message: DEFAULT_ERROR_MESSAGE };
    }
    const empty = operations.length === 0;
    if (!empty) {
      const tempFilePath = await exportArtwork();
      artUrl = await doodleApi.uploadDoodleImage(tempFilePath);
      if (typeof artUrl !== 'string' || !artUrl.trim()) {
        return { ok: false, message: DEFAULT_ERROR_MESSAGE };
      }
    }
  }

  const result = await petStore.saveDoodle(artUrl, petId);
  if (!result || !result.ok) {
    return { ok: false, message: (result && result.message) || DEFAULT_ERROR_MESSAGE };
  }
}

```

```

    if (!petStore.saveDoodleShell(petId, operations)) {
      return { ok: false, message: '画作已上传，但本地编辑记录保存失败，请重试' };
    }
    return { ok: true, petId: String(petId), artUrl, operations, empty };
  } catch (error) {
    return { ok: false, message: errorMessage(error) };
  }
}

module.exports = { saveArtwork };
===== 文件: utils/environment-state.js =====
// 应用内环境状态：不读取定位、不调用天气 API。
// 环境以蛋宝宝身份、陪伴日和本机时段为种子，可复现也便于验收。
const DAY_MS = 24 * 60 * 60 * 1000;
const SEASONS = Object.freeze(['spring', 'summer', 'autumn', 'winter']);
const PERIODS = Object.freeze(['day', 'sunset', 'night']);
const WEATHER_POOLS = Object.freeze({
  spring: Object.freeze(['sunny', 'cloudy', 'rain']),
  summer: Object.freeze(['sunny', 'cloudy', 'storm']),
  autumn: Object.freeze(['sunny', 'rain']),
  winter: Object.freeze(['sunny', 'cloudy', 'snow', 'postSnow'])
});

function timestampOf(value) {
  const timestamp = Number(value);
  return Number.isFinite(timestamp) && timestamp > 0 ? timestamp : Date.now();
}

function localParts(value) {
  const date = new Date(timestampOf(value));
  return {
    year: date.getFullYear(),
    month: date.getMonth() + 1,
    day: date.getDate(),
    hour: date.getHours(),
    minute: date.getMinutes()
  };
}

function localDateKey(value) {
  const parts = localParts(value);
  return [parts.year, String(parts.month).padStart(2, '0'), String(parts.day).padStart(2, '0')].join('-');
}

function localDaySerial(value) {
  const parts = localParts(value);
  return Math.floor(new Date(parts.year, parts.month - 1, parts.day).getTime() / DAY_MS);
}

function companionDay(companionStartedAt, value) {
  const raw = companionStartedAt || '';
  const numeric = Number(raw);
  const started = Number.isFinite(numeric) && numeric > 0 ? numeric : Date.parse(raw);
  if (!Number.isFinite(started)) return 1;
  return Math.max(1, localDaySerial(value) - localDaySerial(started) + 1);
}

function seasonBeforeHatch(companionStartedAt, value) {
  return SEASONS[(companionDay(companionStartedAt, value) - 1) % SEASONS.length];
}

```



```
}

function periodFromLocalTime(value) {
  const parts = localParts(value);
  const minuteOfDay = parts.hour * 60 + parts.minute;
  if (minuteOfDay >= 17 * 60 && minuteOfDay < 19 * 60) return 'sunset';
  if (minuteOfDay >= 6 * 60 && minuteOfDay < 17 * 60) return 'day';
  return 'night';
}

function lightPhaseFromPeriod(period) {
  if (period === 'sunset') return 'sunset';
  if (period === 'night') return 'night';
  return 'midday';
}

function hash32(text) {
  let hash = 2166136261;
  const source = String(text || '');
  for (let index = 0; index < source.length; index += 1) {
    hash ^= source.charCodeAt(index);
    hash = Math.imul(hash, 16777619);
  }
  return hash >>> 0;
}

function weatherForSlot(options) {
  const source = options || {};
  const pool = WEATHER_POOLS[source.season] || WEATHER_POOLS.spring;
  const seed = [
    source.eggId || 'legacy-egg',
    source.dateKey || localDateKey(source.timestamp),
    source.period || periodFromLocalTime(source.timestamp)
  ].join('|');
  return pool[hash32(seed) % pool.length];
}

function weatherAssetName(weather) {
  if (weather === 'sunny') return 'clear';
  if (weather === 'postSnow') return 'post_snow';
  return weather;
}

function sceneKey(season, weather, period) {
  return `${season}_${weatherAssetName(weather)}_${period}`;
}

function nextEnvironmentBoundary(value) {
  const timestamp = timestampOf(value);
  const date = new Date(timestamp);
  const candidates = [0, 6, 17, 19].map(hour => new Date(
    date.getFullYear(), date.getMonth(), date.getDate(), hour, 0, 0, 0
  ).getTime()).filter(candidate => candidate > timestamp + 1000);
  if (candidates.length) return Math.min.apply(null, candidates);
  return new Date(date.getFullYear(), date.getMonth(), date.getDate() + 1, 0, 0, 0, 0).getTime();
}

function millisecondsUntilNextEnvironmentBoundary(value) {
  const timestamp = timestampOf(value);
  return Math.max(1000, nextEnvironmentBoundary(timestamp) - timestamp);
}
```

```

}

function resolve(options) {
  const source = options || {};
  const timestamp = timestampOf(source.timestamp);
  const period = periodFromLocalTime(timestamp);
  const dateKey = localDateKey(timestamp);
  const incubationDay = companionDay(source.hatchStartTime, timestamp);
  const season = seasonBeforeHatch(source.hatchStartTime, timestamp);
  const weather = weatherForSlot({
    eggId: source.petId,
    season,
    period,
    dateKey,
    timestamp
  });
  return {
    season,
    weather,
    period,
    lightPhase: lightPhaseFromPeriod(period),
    dateKey,
    incubationDay,
    sceneKey: sceneKey(season, weather, period)
  };
}

module.exports = {
  resolve,
  periodFromLocalTime,
  seasonBeforeHatch,
  sceneKey,
  millisecondsUntilNextEnvironmentBoundary
};
===== 文件: utils/feedback-api.js =====
const { get, post } = require('./request');

// 诉求类型字典类型编码，与后端 sys_dict_type 对应
const FEEDBACK_TYPE_DICT = 'EGG_FEEDBACK_TYPE';

// 从后端字典拉取诉求类型，SysDictDataItem: [{ name, key }] -> [{ label, value }]
function listFeedbackTypes() {
  return get('/admin/dict/data/type/${FEEDBACK_TYPE_DICT}').then((items) => {
    if (!Array.isArray(items)) return [];
    return items
      .filter((item) => item && item.key && item.name)
      .map((item) => ({ label: item.name, value: item.key }));
  });
}

// 提交反馈，成功返回 { receiptNumber, createDate }
function submitFeedback(payload) {
  return post('/feedback', {
    type: payload.type,
    content: payload.content,
    consent: payload.consent === true
  });
}

module.exports = { listFeedbackTypes, submitFeedback };

```

```

===== 文件: utils/incubation-environment.js =====
// 破壳前环境解析: 场景 key -> OSS 图片 URL 与样式类名。
// 窗景两级查找: sceneKey 精确图优先, 其次 weather×period 共享图;
// 未命中返回空串 (窗户点击无响应), 禁止跨天气/季节或用房间图回退。
const state = require('./environment-state');
const { SCENE_OPTIONS, WINDOW_WEATHER } = require('../config/pre-hatch-assets');

const SCENE_BY_KEY = SCENE_OPTIONS.reduce((map, scene) => {
  map[scene.key] = scene;
  return map;
}, {});

function windowAssetPath(sceneKey, weather, period) {
  const exact = WINDOW_WEATHER.bySceneKey[String(sceneKey || "")];
  if (exact) return exact;
  const byWeather = {
    sunny: { day: WINDOW_WEATHER.clearDay, sunset: WINDOW_WEATHER.clearSunset, night: WINDOW_WEATHER.clearNight },
    cloudy: { day: WINDOW_WEATHER.cloudyDay, night: WINDOW_WEATHER.cloudyNight },
    snow: { day: WINDOW_WEATHER.snowDay, night: WINDOW_WEATHER.snowNight }
  };
  return (byWeather[String(weather || "")] || {})[String(period || "") || ""];
}

function resolveScene(environment) {
  const scene = SCENE_BY_KEY[environment.sceneKey];
  if (!scene) {
    return Object.assign({}, environment, {
      valid: false, fullSceneImage: "", nestImage: "", eggImage: "", windowImage: "",
      className: `season-${environment.season} weather-${environment.weather} period-${environment.period} light-${environment.lightPhase}`
    });
  }
  return Object.assign({}, environment, {
    valid: true,
    fullSceneImage: scene.background,
    nestImage: scene.nest,
    eggImage: scene.egg,
    windowImage: windowAssetPath(environment.sceneKey, environment.weather, environment.period),
    className: scene.className
  });
}

function resolveForPet(pet, timestamp) {
  return resolveScene(state.resolve({
    petId: pet && pet.id,
    hatchStartTime: pet && pet.hatchStartTime,
    timestamp
  }));
}

module.exports = { resolveForPet, resolveScene, windowAssetPath };
===== 文件: utils/invite-api.js =====
const { get } = require('./request');

// GET /invite/mine 返回 InviteCodeVO:
// { code, quota, usedCount, remaining, status, expireTime, createDate }
function getMine() {
  return get('/invite/mine');
}

```

```
module.exports = { getMine };
===== 文件: utils/nfc-claim-api.js =====
const { get, post } = require('./request');

function preview(claimRef) {
  return get('/pdc/nfc/claim/preview', { claimRef });
}

function confirm(claimRef, requestId) {
  return post('/pdc/nfc/claim/confirm', { claimRef, requestId });
}

module.exports = { preview, confirm };
===== 文件: utils/nfc-claim-intent.js =====
const STORAGE_KEY = 'eggbaby_nfc_claim_intent_v1';
const TTL_MS = 30 * 60 * 1000;
const NFC_CLAIM_PATH = 'pages/nfc-claim/nfc-claim';
const CLAIM_REF_PATTERN = /^[A-Za-z0-9_-]{22}$/;

function isNfcClaimPath(path) {
  if (!path) return false;
  const normalized = path.replace(/^\/\//, '');
  return normalized === NFC_CLAIM_PATH;
}

function captureNfcClaimIntent(options, now = Date.now()) {
  if (!isNfcClaimPath(options && options.path)) {
    return getPendingNfcClaimIntent(now);
  }
  const query = options.query || {};
  if (String(query.v) !== '1' || !CLAIM_REF_PATTERN.test(query.ref || '')) {
    clearPendingNfcClaimIntent();
    return null;
  }
  const intent = {
    type: 'NFC_CLAIM',
    version: 1,
    claimRef: query.ref,
    capturedAt: now,
    expiresAt: now + TTL_MS
  };
  wx.setStorageSync(STORAGE_KEY, intent);
  return intent;
}

function getPendingNfcClaimIntent(now = Date.now()) {
  try {
    const intent = wx.getStorageSync(STORAGE_KEY);
    if (!intent || intent.expiresAt <= now) {
      clearPendingNfcClaimIntent();
      return null;
    }
    return intent;
  } catch (e) {
    clearPendingNfcClaimIntent();
    return null;
  }
}
```

```

function clearPendingNfcClaimIntent() {
  try { wx.removeStorageSync(STORAGE_KEY); } catch (e) { /* ignore */ }
}

module.exports = {
  captureNfcClaimIntent,
  getPendingNfcClaimIntent,
  clearPendingNfcClaimIntent,
  isNfcClaimPath,
  STORAGE_KEY,
  TTL_MS
};
===== 文件: utils/ota.js =====
/**
 * utils/ota.js
 * -----
 * OTA helper for the egg-miniprogram chat channel.
 *
 * Calls POST /xiaozhi/ota/ directly with raw wx.request, because the OTA
 * endpoint returns a raw DeviceReportRespDTO (not the egg request.js
 * {code,msg,data} envelope). Resolves the websocket url + token for the
 * active pet's ai_device.id.
 * -----
 */

const { API_BASE_URL } = require('../config/api');

function createError(message) {
  return { type: 'ota', message, userMessage: message };
}

/**
 * Check OTA for the active pet's device and return websocket credentials.
 * @param {string} deviceId - active pet's ai_device.id (macAddress)
 * @returns {Promise<{wsUrl:string, wsToken:string}>}
 */
function checkOrRegisterDevice(deviceId) {
  return new Promise((resolve, reject) => {
    if (!deviceId) {
      reject(createError('缺少宠物设备标识'));
      return;
    }
  });
}

const app = getApp();
const version = (app && app.globalData && app.globalData.version) || '1.0.0-mvp';

wx.request({
  url: `${API_BASE_URL}/ota/`,
  method: 'POST',
  data: {
    application: {
      name: 'xiaozhi-egg-miniprogram',
      version,
    },
    board: {
      type: 'wechat-egg-miniprogram',
      mac: deviceId,
    },
    chip_model_name: 'wechat-egg-miniprogram',
  },
});

```

```

header: {
  'Device-ID': deviceId,
  'Client-ID': 'wechat-miniprogram',
},
success: (response) => {
  const statusCode = response && response.statusCode;
  const data = response && response.data;

  if (typeof statusCode !== 'number' || statusCode < 200 || statusCode >= 300) {
    reject(createError('获取聊天配置失败: ' + statusCode));
    return;
  }

  if (!data || !data.websocket) {
    reject(createError('聊天配置不可用，请确认蛋宝宝已破壳'));
    return;
  }

  resolve({
    wsUrl: data.websocket.url,
    wsToken: data.websocket.token,
  });
},
fail: () => {
  reject(createError('网络异常，暂时无法获取聊天配置'));
},
});
});
}

module.exports = { checkOrRegisterDevice };
===== 文件: utils/pet-api.js =====
const { get, post, put } = require('./request');

// 后端 PetVO 字段: id, userId, deviceId, nickname, birthDate, bazi, wuxing, zodiac,
// mbti, personality, personalityBrief, todayMood, todayMoodDate, todayMoodSentence,
// hatchStatus(EGG/HATCHED), hatchStartTime, expectedHatchTime, hatchedAt, acceleratedMinutes,
// avatarUrl, prototype, gender, bloodType, createDate
function adoptPet(inviteCode) {
  return post('/pet/adopt', { inviteCode });
}

// HatchActionResultVO: { addedMinutes, alreadyDone, readyToHatch, pet }
function submitHatchAction(petId, type, payload) {
  return post(`/pet/${petId}/hatch-action`, { type, payload });
}

// HatchActionVO[]: [{ id, actionType, payload, actionDate, acceleratedMinutes, createDate }]
function listHatchActions(petId) {
  return get(`/pet/${petId}/hatch-actions`);
}

function hatchPet(petId) {
  return post(`/pet/${petId}/hatch`);
}

function getPet(petId) {
  return get(`/pet/${petId}`);
}

```

```

function listPets() {
  return get('/pet/list');
}

function updateNickname(petId, nickname) {
  return put('/pet/update', { id: petId, nickname });
}

function listChatHistory(agentId, deviceId, page, limit) {
  return get('/agent/chat-history/list', {
    agentId,
    macAddress: deviceId,
    page: page || 1,
    limit: limit || 20
  });
}

// 每日用户聊天依赖提醒状态: { todayCount, minor, chatLimited }
// todayCount 为当日用户发送消息数 (chat_type=1, Asia/Shanghai 日界);
// chatLimited=true 表示未成年人 (≤14 周岁) 当日已超阈值, 前端需弹窗并退出聊天页。
function getDailyUserChatCount() {
  return get('/agent/chat-history/daily-user-count');
}

module.exports = {
  adoptPet,
  submitHatchAction,
  listHatchActions,
  hatchPet,
  getPet,
  listPets,
  updateNickname,
  listChatHistory,
  getDailyUserChatCount
};
===== 文件: utils/pet-store.js =====
const petApi = require('./pet-api');
const PET_KEY = 'eggbaby_mvp_pet_v1';
const USER_KEY = 'eggbaby_mvp_user_v1';
const IDENTITY_KEY = 'eggbaby_mvp_identity_v1';
const ACTIVE_PET_KEY = 'eggbaby_active_pet_v1';
// 涂鸦操作序列(shell)本地缓存: 仅存操作 JSON, 用于重开编辑器恢复画布继续编辑; 云端只存合成 PNG
const DOODLE_SHELL_KEY = 'eggbaby_doodle_shell_v1';

const DAY = 24 * 60 * 60 * 1000;
const HATCH_TOTAL_MINUTES = 7 * 24 * 60;

const STATUS_LINES = {
  egg: {
    开心: ['蛋壳里传来轻轻的回应。','它把你的声音藏进了壳里。'],
    平静: ['蛋壳里很安静, 但很暖。','它今天睡得很踏实。'],
    想念: ['它好像等了你很久。','它把想你藏在壳里。'],
    兴奋: ['蛋壳里的动静变多了。','裂纹好像又亮了一点。'],
    低落: ['它今天安静了很久。','蛋壳里的声音变小了。'],
  },
  pet: {
    开心: ['它把快乐摆在了脸上。','它好像一直在等你来。'],
    平静: ['它今天把日子过得很慢。','它安静地待在你身边。'],
    想念: ['它偷偷练习了怎么叫你。','它把想你写进了今天。'],
    兴奋: ['它好像准备了一个小秘密。','它今天比平时更坐不住。'],
  }
}

```

```

    低落: ['它今天有一点点没精神。', '它安静了很久，像是在等你。']
  }
};

function todayKey(now) {
  const date = now ? new Date(now) : new Date();
  const year = date.getFullYear();
  const month = String(date.getMonth() + 1).padStart(2, '0');
  const day = String(date.getDate()).padStart(2, '0');
  return `${year}-${month}-${day}`;
}

function read(key) {
  try { return wx.getStorageSync(key) || null; } catch (error) { return null; }
}

function write(key, value) {
  try { wx.setStorageSync(key, value); } catch (error) {}
  return value;
}

function getUser() {
  return read(USER_KEY);
}

function saveUser(user) {
  if (user && user.id) write(IDENTITY_KEY, user.id);
  return write(USER_KEY, user);
}

/**
 * 把后端用户资料的 nickname/avatarUrl/ageRange 同步到本地 USER_KEY 缓存。
 * ageRange 供聊天页年龄区间强制门槛读取。
 * @param {Object} profile GET /wechat/profile 响应
 */
function syncUserProfile(profile) {
  if (!profile) return;
  const user = getUser() || {};
  if (profile.nickname !== undefined) user.nickname = profile.nickname;
  if (profile.avatarUrl !== undefined) user.avatarUrl = profile.avatarUrl;
  if (profile.ageRange !== undefined) user.ageRange = profile.ageRange;
  saveUser(user);
}

function getIdentityId() {
  return read(IDENTITY_KEY);
}

// 账号级缓存 key，登出/换账号时必须全部清除，防止上一个账号的数据泄漏给下一个账号。
// 后两个 key 分别由 pages/profile 与 pages/deregister 写入，这里统一纳入清理。
const ACCOUNT_KEYS = [
  PET_KEY, USER_KEY, IDENTITY_KEY, ACTIVE_PET_KEY,
  DOODLE_SHELL_KEY,
  'eggbaby_profile_v1',
  'eggbaby_deregister_request_v1'
];

function clearAccountData() {
  ACCOUNT_KEYS.forEach((key) => {
    try { wx.removeStorageSync(key); } catch (error) {}
  });
}

```



```

});
}

function clearUser() {
  try { wx.removeStorageSync(USER_KEY); } catch (error) {}
}

function getPet() {
  const pet = read(PET_KEY);
  const user = getUser();
  if (pet && pet.ownerId && user && pet.ownerId !== user.id) return null;
  return pet;
}

function savePet(pet) {
  return write(PET_KEY, pet);
}

function isBound() {
  return !!getPet();
}

function toTimestamp(value) {
  if (!value) return null;
  const normalized = typeof value === 'string' ? value.trim().replace(' ', 'T') : value;
  const ms = new Date(normalized).getTime();
  return Number.isFinite(ms) ? ms : null;
}

// 把后端 PetVO 映射成本地 pet 形状并缓存。
// 非 demo 修炼动作已接后端：后端 PetVO 不返回前端独占字段(shell/tasks/preferences/messages/inviteCodes/
  dailyStatus),
// 故这些字段从已缓存 pet 合并(首次领养无缓存时回退默认值)，避免每次 action 调用清空本地状态。
// 后端派生字段(id/hatchStatus/acceleratedMinutes/时间戳/身份字段/todayMood/deviceId)以 vo 为唯一事实源
。
// stage/进度/倒计时由 getStage/getCountdown 从 hatchStatus/hatchAt/acceleratedMinutes 派生。
function savePetFromVO(vo) {
  if (!vo || !vo.id) return null;
  const user = getUser();
  const existing = read(PET_KEY);
  const accelerated = vo.acceleratedMinutes || 0;
  const progress = Math.max(0, Math.min(100, Math.round((accelerated / HATCH_TOTAL_MINUTES) * 100)));
  const hatchStartTime = toTimestamp(vo.hatchStartTime);
  const hatchAt = toTimestamp(vo.expectedHatchTime)
    || (hatchStartTime ? hatchStartTime + 7 * DAY : Date.now() + 7 * DAY);
  const createdAt = toTimestamp(vo.createDate) || hatchStartTime || Date.now();
  const isHatched = vo.hatchStatus === 'HATCHED';
  const pet = {
    id: vo.id,
    ownerId: (user && user.id) || null,
    prototype: vo.prototype || '玉兔',
    name: vo.nickname || (existing ? existing.name : '') || '',
    createdAt,
    hatchAt,
    progress,
    stage: 'waiting',
    // 保留本地 recordTouch 记录的互动时间，避免每次后端刷新都重置回 createdAt,
    // 否则老宠走本地 fallback 心情时 inactiveDays 恒为巨大值 → 永远"低落"
    lastInteractionAt: Math.max((existing && existing.lastInteractionAt) || 0, createdAt),
    tasks: existing && existing.tasks !== undefined ? existing.tasks : { nicknameDone: false, cuddleDate

```

```

: ", wishDate: ", lessonDate: ", doodleDone: false },
preferences: existing && existing.preferences !== undefined ? existing.preferences : { wishes: [], lessons: [] },
shell: existing && existing.shell !== undefined ? existing.shell : { color: '#EDE78E', colorName: '奶油白', pattern: '星星' },
dailyStatus: existing && existing.dailyStatus !== undefined ? existing.dailyStatus : null,
inviteCodes: existing && existing.inviteCodes !== undefined ? existing.inviteCodes : [],
messages: existing && existing.messages !== undefined ? existing.messages : [],
todayMood: vo.todayMood || "",
todayMoodSentence: vo.todayMoodSentence || "",
todayMoodDate: vo.todayMoodDate || "",
hatchStatus: vo.hatchStatus || 'EGG',
acceleratedMinutes: accelerated,
hatchStartTime: toTimestamp(vo.hatchStartTime),
expectedHatchTime: toTimestamp(vo.expectedHatchTime),
hatchedAt: toTimestamp(vo.hatchedAt),
deviceId: vo.deviceId || null,
agentId: vo.agentId || null,
bazi: vo.bazi || "",
wuxing: vo.wuxing || "",
zodiac: translateZodiac(vo.zodiac) || "",
mbti: vo.mbti || "",
personality: vo.personality || "",
personalityBrief: vo.personalityBrief || "",
gender: vo.gender || "",
bloodType: vo.bloodType || "",
avatarUrl: vo.avatarUrl || "",
collectionCards: vo.collectionCards || [],
sceneUrl: vo.sceneUrl || ""
};
if (existing && Array.isArray(existing._hatchActions)) pet._hatchActions = existing._hatchActions;
savePet(pet);
setActivePetId(pet.id);
return pet;
}

function getActivePetId() {
  return read(ACTIVE_PET_KEY);
}

function setActivePetId(id) {
  return write(ACTIVE_PET_KEY, id);
}

function mockCodeError(code) {
  const value = code.toUpperCase();
  const errors = {
    INVALID: '激活码无效，请检查后重试',
    USED: '该激活码已被使用',
    FULL: '该激活码名额已满',
    PAUSED: '该激活码暂不可用'
  };
  return errors[value] || "";
}

function createInviteCode(seed) {
  // 与后端 ai_invite_code 单码配额模型对齐：个人仅 1 个邀请码，有总配额/已用/剩余。
  // 此处为 mock 演示值（1 已用 / 5 配额），真实数据由 GET /invite/mine 返回。
  const suffix = String(seed).slice(-4);
  const quota = 5;

```

```
const usedCount = 1;
return {
  code: `EGG-${suffix}-X`,
  quota,
  usedCount,
  remaining: quota - usedCount,
  status: 1
};
}

function bindPet(code, now) {
  if (isBound()) {
    return { ok: false, reason: 'BOUND', message: '当前版本一个账号只能绑定 1 只蛋宝宝，本次激活码未被消耗' };
  }
  const normalized = String(code || '').trim().toUpperCase();
  if (!normalized) return { ok: false, reason: 'EMPTY', message: '请输入激活码' };
  const error = mockCodeError(normalized);
  if (error) return { ok: false, reason: normalized, message: error };

  const createdAt = now || Date.now();
  const prototype = normalized.includes('KOI') ? '锦鲤' : '玉兔';
  const hatchAt = normalized === 'HATCH-NOW' ? createdAt : createdAt + 7 * DAY;
  const id = `egg-${createdAt}`;
  const pet = {
    id,
    ownerId: (getUser() && getUser().id) || '',
    prototype,
    name: '',
    createdAt,
    hatchAt,
    progress: 0,
    stage: 'waiting',
    lastInteractionAt: createdAt,
    tasks: {
      nicknameDone: false,
      cuddleDate: '',
      wishDate: '',
      lessonDate: '',
      doodleDone: false
    },
    preferences: { wishes: [], lessons: [] },
    shell: { color: '#EDE78E', colorName: '奶油白', pattern: '星星' },
    dailyStatus: null,
    collectionCards: [],
    inviteCode: createInviteCode(createdAt),
    messages: []
  };
  savePet(pet);
  return { ok: true, pet };
}

function addProgress(pet, amount) {
  pet.progress = Math.min(100, pet.progress + amount);
  pet.stage = pet.progress > 0 ? 'hatching' : 'waiting';
  return pet;
}

async function updateNickname(name) {
  const pet = getPet();
```

```

if (!pet) return { ok: false, message: '还没有蛋宝宝' };
const value = String(name || '').trim();
if (!value) return { ok: false, message: '昵称不能为空' };
if (Array.from(value).length > 10) return { ok: false, message: '昵称最多 10 个字符' };
if (['违法', '诈骗', '赌博'].some(word => value.includes(word))) return { ok: false, message: '昵称含有不适合的内容，请换一个' };
try {
  let updated;
  if (pet.hatchStatus === 'HATCHED') {
    // 破壳后 NICKNAME 修炼动作不再合法(后端抛 10209)，改名直接走 PUT /pet/update
    const vo = await petApi.updateNickname(pet.id, value);
    updated = savePetFromVO(vo);
  } else {
    const result = await petApi.submitHatchAction(pet.id, 'NICKNAME', { nickname: value });
    if (result.alreadyDone) {
      // 后端 HatchActionService 仅在首次提交时持久化 nickname；二次提交走 PUT /pet/update 兜底。
      const vo = await petApi.updateNickname(pet.id, value);
      updated = savePetFromVO(vo);
    } else {
      updated = savePetFromVO(result.pet);
    }
  }
  updated.name = value;
  savePet(updated);
  return { ok: true, alreadyDone: pet.hatchStatus === 'HATCHED', pet: updated };
} catch (error) {
  return { ok: false, message: (error && error.userMessage) || '提交失败，请稍后重试' };
}
}

const ACTION_TYPE = { cuddle: 'CUDDLE', wish: 'WISH', lesson: 'LESSON' };

async function completeDailyTask(task, value) {
  const pet = getPet();
  if (!pet) return { ok: false, message: '还没有蛋宝宝' };

  // 许愿支持对象入参 { questionId, value }，兼容旧字符串入参
  var wishData = null;
  if (task === 'wish') {
    if (typeof value === 'object' && value !== null) {
      wishData = { questionId: value.questionId || 'legacy', value: value.value || '' };
    } else {
      wishData = { questionId: 'legacy', value: String(value || '') };
    }
  }

  try {
    const payload = task === 'cuddle' ? {} : (task === 'wish' ? { questionId: wishData.questionId, value: wishData.value } : { value });
    const result = await petApi.submitHatchAction(pet.id, ACTION_TYPE[task], payload);
    const updated = savePetFromVO(result.pet);
    if (!result.alreadyDone) {
      if (task === 'wish') updated.preferences.wishes.push({ date: todayKey(), questionId: wishData.questionId, value: wishData.value });
      if (task === 'lesson') updated.preferences.lessons.push({ date: todayKey(), value });
    }
    savePet(updated);
    return { ok: true, alreadyDone: !result.alreadyDone, pet: updated };
  } catch (error) {
    return { ok: false, message: (error && error.userMessage) || '提交失败，请稍后重试' };
  }
}

```

```
}
}

function completeCuddle() {
  return completeDailyTask('cuddle', '贴贴');
}

function completeWish(value) {
  return completeDailyTask('wish', value);
}

function completeLesson(value) {
  return completeDailyTask('lesson', value);
}

async function saveDoodle(artUrl, petId) {
  const pet = getPet();
  if (!pet) return { ok: false, message: '还没有蛋宝宝' };
  if (!petId || String(pet.id) !== String(petId)) {
    return { ok: false, message: '蛋宝宝状态已变化，请返回后重试' };
  }
  try {
    const result = await petApi.submitHatchAction(petId, 'DOODLE', { artUrl });
    const currentPet = getPet();
    const responsePet = result && result.pet;
    if (!currentPet || String(currentPet.id) !== String(petId)
      || !responsePet || String(responsePet.id) !== String(petId)) {
      return { ok: false, message: '蛋宝宝状态已变化，请返回后重试' };
    }
    const updated = savePetFromVO(result.pet);
    savePet(updated);
    return { ok: true, alreadyDone: !!result.alreadyDone, pet: updated, petId: String(petId) };
  } catch (error) {
    return { ok: false, message: (error && error.userMessage) || '提交失败，请稍后重试' };
  }
}

// 涂鸦操作序列(shell)写入本地缓存，按 petId 关联；返回值用于保存事务确认本地记录已落盘
function saveDoodleShell(petId, operations) {
  if (!petId || !Array.isArray(operations)) return false;
  try {
    wx.setStorageSync(DOODLE_SHELL_KEY, { petId, operations, savedAt: Date.now() });
    return true;
  } catch (error) {
    return false;
  }
}

// 读取本地涂鸦操作序列；petId 不匹配(换了宠物/数据串号)视为无缓存返回 null
function getDoodleShell(petId) {
  const record = read(DOODLE_SHELL_KEY);
  if (!record || record.petId !== petId || !Array.isArray(record.operations)) return null;
  return record.operations;
}

// 修炼任务完成态：无 _hatchActions 缓存时从 pet.tasks 读取；有缓存则按当日派生。
function getHatchActionState(pet) {
  if (!pet) return { nicknameDone: false, cuddleDone: false, wishDone: false, lessonDone: false, doodleDone: false };
  if (!Array.isArray(pet._hatchActions) || pet._hatchActions.length === 0) {
```

```

const today = todayKey();
return {
  nicknameDone: !(pet.tasks && pet.tasks.nicknameDone),
  cuddleDone: !(pet.tasks && pet.tasks.cuddleDate === today),
  wishDone: !(pet.tasks && pet.tasks.wishDate === today),
  lessonDone: !(pet.tasks && pet.tasks.lessonDate === today),
  doodleDone: !(pet.tasks && pet.tasks.doodleDone)
};
}
const today = todayKey();
const done = {};
pet._hatchActions.forEach((a) => {
  if (a.actionDate === today || a.actionType === 'NICKNAME' || a.actionType === 'DOODLE') {
    done[a.actionType] = true;
  }
});
return {
  nicknameDone: !!done.NICKNAME,
  cuddleDone: !!done.CUDDLE,
  wishDone: !!done.WISH,
  lessonDone: !!done.LESSON,
  doodleDone: !!done.DOODLE
};
}

function getStage(pet, now) {
  if (!pet) return 'empty';
  if (pet.hatchStatus === 'HATCHED') return 'hatched';
  const current = now || Date.now();
  if (current >= pet.hatchAt) return 'ready';
  if (pet.hatchAt - current <= DAY) return 'soon';
  // 单轨 Model X: 进度满即时间到 (落到 ready), 不保留 prepared 中间态
  return pet.acceleratedMinutes > 0 || pet.progress > 0 ? 'hatching': 'waiting';
}

const STAGE_PRESENTATION = {
  waiting: { homeText: '它还在睡觉, 试着叫醒它吧', actionLabel: '孵化修炼手册', myStage: '待激活' },
  hatching: { homeText: '它正在慢慢长大', actionLabel: '孵化修炼手册', myStage: '孵化中' },
  soon: { homeText: '蛋壳里传来了动静', actionLabel: '孵化修炼手册', myStage: '即将破壳' },
  ready: { homeText: '它准备好见你了', actionLabel: '查看破壳结果', myStage: '待破壳' },
  hatched: { homeText: '它终于来到你身边了', actionLabel: '和它说说话', myStage: '已破壳' }
};

function getStagePresentation(stage) {
  return STAGE_PRESENTATION[stage] || STAGE_PRESENTATION.waiting;
}

function getCountdown(pet, now) {
  if (!pet) return '';
  const remaining = pet.hatchAt - (now || Date.now());
  if (remaining <= 0) return '破壳时刻已到';
  const days = Math.floor(remaining / DAY);
  const hours = Math.floor((remaining % DAY) / (60 * 60 * 1000));
  return days > 0 ? `还剩 ${days} 天 ${hours} 小时` : `还剩 ${hours} 小时`;
}

function simpleHash(value) {
  return Array.from(String(value)).reduce((sum, char) => sum + char.charCodeAt(0), 0);
}

```

```

_ellipse(ctx, x, y, rx, ry, rotation) {
  ctx.save();
  ctx.translate(x, y);
  ctx.rotate(rotation);
  ctx.scale(rx, ry);
  ctx.beginPath();
  ctx.arc(0, 0, 1, 0, Math.PI * 2);
  ctx.restore();
},

_wrapText(ctx, text, maxWidth, maxLines) {
  const chars = String(text || '').split("");
  const lines = [];
  let line = "";
  for (const ch of chars) {
    const test = line + ch;
    if (ctx.measureText(test).width > maxWidth && line) {
      lines.push(line);
      line = ch;
      if (lines.length >= maxLines) break;
    } else {
      line = test;
    }
  }
  if (line && lines.length < maxLines) lines.push(line);
  return lines;
},

_drawPetAvatar(ctx, type, x, y, size) {
  ctx.save();
  ctx.translate(x, y);
  const s = size;

  if (type === '玉兔') {
    // 耳朵 (z-index 在 face 之下, 先绘制)
    // 左耳
    ctx.save();
    ctx.translate(s * 0.28, s * 0.18);
    ctx.rotate(-9 * Math.PI / 180);
    ctx.beginPath();
    this._ellipse(ctx, 0, 0, s * 0.12, s * 0.28, 0);
    ctx.setFillStyle('#FFF9E4');
    ctx.fill();
    ctx.beginPath();
    this._ellipse(ctx, 0, s * 0.04, s * 0.05, s * 0.18, 0);
    ctx.setFillStyle('rgba(244,185,174,0.7)');
    ctx.fill();
    ctx.restore();

    // 右耳
    ctx.save();
    ctx.translate(s * 0.72, s * 0.18);
    ctx.rotate(9 * Math.PI / 180);
    ctx.beginPath();
    this._ellipse(ctx, 0, 0, s * 0.12, s * 0.28, 0);
    ctx.setFillStyle('#FFF9E4');
    ctx.fill();
    ctx.beginPath();
    this._ellipse(ctx, 0, s * 0.04, s * 0.05, s * 0.18, 0);
    ctx.setFillStyle('rgba(244,185,174,0.7)');
  }
}

```

```
ctx.fill();
ctx.restore();

// 脸部 (z-index 在耳朵之上)
ctx.beginPath();
this._ellipse(ctx, s / 2, s * 0.55, s * 0.38, s * 0.32, 0);
ctx.setFillStyle('#FFF9E4');
ctx.fill();

// 眼睛
ctx.beginPath();
ctx.arc(s * 0.38, s * 0.52, s * 0.04, 0, Math.PI * 2);
ctx.arc(s * 0.62, s * 0.52, s * 0.04, 0, Math.PI * 2);
ctx.setFillStyle('#002900');
ctx.fill();

// 鼻子
ctx.beginPath();
this._ellipse(ctx, s / 2, s * 0.58, s * 0.04, s * 0.035, 0);
ctx.setFillStyle('#D8908B');
ctx.fill();

// 腮红
ctx.beginPath();
this._ellipse(ctx, s * 0.30, s * 0.62, s * 0.06, s * 0.04, 0);
this._ellipse(ctx, s * 0.70, s * 0.62, s * 0.06, s * 0.04, 0);
ctx.setFillStyle('rgba(244,185,174,0.45)');
ctx.fill();
} else {
// 锦鲤
ctx.save();
ctx.translate(s / 2, s / 2);
ctx.rotate(-7 * Math.PI / 180);
ctx.translate(-s / 2, -s / 2);

// 尾巴
ctx.save();
ctx.translate(s * 0.83, s * 0.50);
ctx.rotate(45 * Math.PI / 180);
ctx.beginPath();
this._ellipse(ctx, 0, 0, s * 0.17, s * 0.28, 0);
ctx.setFillStyle('#F4B9AE');
ctx.fill();
ctx.restore();

// 身体 (z-index 在 tail 之上、fin 之下)
ctx.beginPath();
this._ellipse(ctx, s * 0.48, s * 0.54, s * 0.36, s * 0.31, 0);
ctx.setFillStyle('#FFF9E4');
ctx.fill();

// 鱼鳍 (z-index 在身体之上)
ctx.save();
ctx.translate(s * 0.55, s * 0.74);
ctx.rotate(18 * Math.PI / 180);
ctx.beginPath();
this._ellipse(ctx, 0, 0, s * 0.15, s * 0.13, 0);
ctx.setFillStyle('#EDE78E');
ctx.fill();
ctx.restore();
```



```

// 眼睛
ctx.beginPath();
ctx.arc(s * 0.28, s * 0.46, s * 0.04, 0, Math.PI * 2);
ctx.setFillStyle('#002900');
ctx.fill();

// 斑点
ctx.save();
ctx.translate(s * 0.53, s * 0.30);
ctx.rotate(22 * Math.PI / 180);
ctx.beginPath();
this._ellipse(ctx, 0, 0, s * 0.15, s * 0.21, 0);
ctx.setFillStyle('#E77E72');
ctx.fill();
ctx.restore();

ctx.beginPath();
this._ellipse(ctx, s * 0.72, s * 0.66, s * 0.115, s * 0.17, 0);
ctx.setFillStyle('#E77E72');
ctx.fill();

ctx.restore();
}

ctx.restore();
},

onAlbum() { wx.navigateTo({ url: '/pages/album/album' }); },

onSave() {
  wx.canvasToTempFilePath({
    canvasId: 'shareCanvas',
    width: 750,
    height: 1280,
    destWidth: 1500,
    destHeight: 2560,
    success: ({ tempFilePath }) => {
      wx.saveImageToPhotosAlbum({
        filePath: tempFilePath,
        success: () => wx.showToast({ title: '收藏卡已保存', icon: 'success' }),
        fail: () => wx.showToast({ title: '请允许保存到相册', icon: 'none' })
      });
    },
    fail: () => wx.showToast({ title: '图片生成失败，请重试', icon: 'none' })
  }, this);
},

onShareAppMessage() {
  const card = this.data.card;
  if (!card || !this.data.shareReady) return false;
  const inviteCode = this._shareInviteCode;
  return {
    title: `我孵化了${card.name}, 编号 ${card.serial}`,
    path: inviteCode ? `${SHARE_HOME_PATH}&inviteCode=${encodeURIComponent(inviteCode)}` : SHARE_HOME_PATH
  };
},

onShareTimeline() {

```

```

const card = this.data.card;
if (!card || !this.data.shareReady) return false;
return { title: `我孵化了${card.name}, 编号 ${card.serial}` };
}
});
===== 文件: pages/collection-card/collection-card.wxml =====
<view class="page">
  <nav-bar title="收藏卡"></nav-bar>
  <scroll-view class="content" scroll-y>
    <view wx:if="{{isNew}}" class="new-banner">获得第一张破壳收藏卡 · 已自动收入卡册</view>
    <view class="card">
      <view class="card-title-section">
        <view class="title-copy">
          <text class="name">{{card.name}}</text>
          <text class="collect-context">{{subtitle}}</text>
        </view>
      </view>
      <view class="card-illustration-section">
        <image wx:if="{{card.imageUrl}}" class="illustration" src="{{card.imageUrl}}" mode="aspectFill"
show-menu-by-longpress></image>
        <pet-avatar wx:else pet-type="{{card.petType}}" size="{{250}}"></pet-avatar>
      </view>
      <view class="card-data-section">
        <view class="stat-grid">
          <view class="stat-cell"><text class="stat-label">类型</text><text class="stat-value">{{card.pe
tType}}</text></view>
          <view class="stat-cell"><text class="stat-label">生日</text><text class="stat-value">{{birthda
yLabel}}</text></view>
          <view class="stat-cell"><text class="stat-label">星座</text><text class="stat-value stat-value
--symbol">{{card.zodiac}} {{zodiacSymbol}}</text></view>
          <view class="stat-cell">
            <text class="stat-label">性别</text>
            <view wx:if="{{genderClass}}" class="gender-icon {{genderClass}}" aria-label="{{genderLabel}}
"></view>
            <text wx:else class="stat-value">{{genderLabel}}</text>
          </view>
          <view class="stat-cell"><text class="stat-label">血型</text><text class="stat-value">{{card.bl
oodType}} 型</text></view>
          <view class="stat-cell"><text class="stat-label">MBTI</text><text class="stat-value">{{card.mb
ti}}</text></view>
        </view>
        <text class="card-signature {{signatureClass}}">{{card.personality}}</text>
      </view>
    </view>
    <view class="actions">
      <view class="action-row">
        <button class="secondary" bindtap="onSave">保存图片</button>
        <button wx:if="{{shareReady}}" class="secondary" open-type="share">分享给好友</button>
      </view>
    </view>
    <canvas canvas-id="shareCanvas" class="share-canvas"></canvas>
  </scroll-view>
</view>
===== 文件: pages/welcome/welcome.js =====
const auth = require('../../utils/auth');
const { getPendingNfcClaimIntent } = require('../../utils/nfc-claim-intent');
const shareInvite = require('../../utils/share-invite');

Page({
  data: {

```

```

    ready: false,
    hasPendingInvite: false
  },

  async onLoad() {
    const hasPendingInvite = Boolean(shareInvite.getPending());
    // 同步检查本地登录态: 已绑定会话直接跳转首页
    const cached = auth.getSession();
    if (cached && !auth.isExpired()) {
      // 有 NFC 领取意图时优先进入领取页, 不要求已绑定手机号
      if (this.navigateNfcClaim()) return;
      if (cached.hasPhone === true) {
        wx.switchTab({ url: '/pages/home/home' });
        return;
      }
      this.setData({ ready: true, hasPendingInvite });
      return;
    }
    // 本地无有效登录态时渲染欢迎页内容并尝试静默登录
    this.setData({ ready: true, hasPendingInvite });
    const session = await getApp().ensureLogin().catch(() => null);
    if (session && session.userId) {
      // 静默登录后同样先恢复 NFC 领取, 再按手机号状态分流
      if (this.navigateNfcClaim()) return;
      if (session.hasPhone === true) {
        wx.switchTab({ url: '/pages/home/home' });
      }
    }
  },

```

```

  navigateNfcClaim() {
    const intent = getPendingNfcClaimIntent();
    if (intent && intent.claimRef) {
      getApp().globalData.welcomeCompleted = true;
      wx.redirectTo({ url: '/pages/nfc-claim/nfc-claim' });
      return true;
    }
    return false;
  },

```

```

  onEnterIsland() {
    // 进入小岛前先恢复 NFC 领取
    if (this.navigateNfcClaim()) return;
    getApp().globalData.welcomeCompleted = true;
    wx.switchTab({ url: '/pages/home/home' });
  }
});
===== 文件: pages/welcome/welcome.wxml =====
<view class="page">
  <view wx:if="{{!ready}}" class="loading">
    <view class="loading-spinner"></view>
  </view>

```

```

  <block wx:if="{{ready}}">
    <view class="halo halo--one"></view>
    <view class="halo halo--two"></view>

    <view class="brand">
      <text class="brand-kicker">EGGBABY</text>
      <text class="brand-name">蛋宝宝</text>
    </view>
  </block>

```

```

<text class="brand-slogan">一颗蛋，一段只属于你的陪伴。</text>
<text wx:if="{{hasPendingInvite}}" class="brand-slogan">好友邀请你一起开启陪伴</text>
</view>

<view class="egg-stage">
  <view class="egg-shadow"></view>
  <view class="egg">
    <view class="egg-speck egg-speck--one"></view>
    <view class="egg-speck egg-speck--two"></view>
    <view class="egg-shine"></view>
  </view>
</view>

<view class="bottom-panel page-safe-bottom">
  <button class="auth-button" bindtap="onEnterIsland">进入快乐仙岛</button>
  <text class="auth-note">开启一段只属于你的陪伴</text>
</view>
</block>
</view>
===== 文件: pages/feedback/feedback.js =====
const feedbackApi = require('../../utils/feedback-api');
const { FALLBACK_FEEDBACK_TYPES } = require('../../config/feedback-types');

const MIN_SUBMITTING_VISIBLE_MS = 600;
// 客服热线：老年人人工反馈渠道，知情同意书弹窗展示与拨号共用
const HOTLINE = '18201931204';

function buildTypeOptions(types) {
  return [{ value: '', label: '请选择诉求类型' }].concat(types);
}

Page({
  data: {
    typeOptions: buildTypeOptions(FALLBACK_FEEDBACK_TYPES),
    typeIndex: 0,
    type: '',
    description: '',
    consent: false,
    typePanelOpen: false,
    submitting: false,
    submitError: '',
    receiptNumber: '',
    canSubmit: false,
    consentVisible: false,
    hotline: HOTLINE
  },

  onLoad() {
    this.loadFeedbackTypes();
  },

  onUnload() {
    this.submissionInFlight = false;
    this.submitRequestToken = (Number(this.submitRequestToken) || 0) + 1;
    clearTimeout(this.submitDelayTimer);
    this.submitDelayTimer = null;
  },

  // 诉求类型来自后端字典；拉取失败沿用本地兜底列表
  loadFeedbackTypes() {

```

```
feedbackApi.listFeedbackTypes()
  .then((types) => {
    if (types.length > 0) {
      this.setData({ typeOptions: buildTypeOptions(types) });
    }
  })
  .catch(() => {});
},

refreshCanSubmit(fields) {
  const next = Object.assign({}, this.data, fields || {});
  this.setData(Object.assign({}, fields || {}, {
    canSubmit: Boolean(next.type && String(next.description || "").trim() && next.consent && !next.submitting)
  }));
},

onToggleTypePanel() {
  if (this.data.submitting) return;
  this.setData({ typePanelOpen: !this.data.typePanelOpen, submitError: "" });
},

onSelectType(event) {
  if (this.data.submitting) return;
  const typeIndex = Number(event.currentTarget.dataset.index);
  const option = this.data.typeOptions[typeIndex];
  if (!option || !option.value) return;
  this.refreshCanSubmit({ typeIndex, type: option.value, typePanelOpen: false, submitError: "" });
},

onDescriptionInput(event) {
  this.refreshCanSubmit({ description: String(event.detail.value || ""), submitError: "" });
},

onToggleConsent() {
  if (this.data.submitting) return;
  this.refreshCanSubmit({ consent: !this.data.consent, submitError: "" });
},

onOpenConsent() {
  this.setData({ consentVisible: true });
},

onCloseConsent() {
  this.setData({ consentVisible: false });
},

// 弹窗内明确点击"同意"才勾选，查看行为本身不构成同意
onAgreeConsent() {
  this.refreshCanSubmit({ consent: true, consentVisible: false, submitError: "" });
},

onCallHotline() {
  wx.makePhoneCall({ phoneNumber: HOTLINE });
},

// 拦截弹窗遮罩的触摸穿透与点击冒泡，无实际逻辑
noop() {},

onSubmit() {
```

```

if (this.data.submitting || this.submissionInFlight) return;
if (!this.data.canSubmit) {
  let submitError = "";
  if (!this.data.type) submitError = '请选择诉求类型';
  else if (!String(this.data.description || "").trim()) submitError = '请填写问题详细描述';
  else if (!this.data.consent) submitError = '请勾选同意知情同意书';
  else submitError = '请完善必填信息后提交';
  this.setData({ submitError });
  wx.showToast({ title: submitError, icon: 'none' });
  return;
}
const payload = {
  type: this.data.type,
  content: this.data.description.trim(),
  consent: true
};
this.submissionInFlight = true;
this.refreshCanSubmit({ submitting: true, submitError: "" });
const requestToken = (Number(this.submitRequestToken) || 0) + 1;
this.submitRequestToken = requestToken;
const minimumVisibleDelay = new Promise(resolve => {
  this.submitDelayTimer = setTimeout(() => {
    this.submitDelayTimer = null;
    resolve();
  }, MIN_SUBMITTING_VISIBLE_MS);
});
const submission = feedbackApi.submitFeedback(payload)
  .then((data) => ({ ok: true, receiptNumber: data.data.receiptNumber }))
  .catch((error) => ({ ok: false, message: (error.data?.errorMessage || '提交失败，请重试' ))));
Promise.all([submission, minimumVisibleDelay]).then(([result]) => {
  if (this.submitRequestToken !== requestToken) return;
  this.submissionInFlight = false;
  if (!result || !result.ok) {
    this.refreshCanSubmit({ submitting: false, submitError: result.data?.errorMessage || '提交失败，请重试' });
    return;
  }
  this.setData({
    typeIndex: 0,
    type: "",
    description: "",
    consent: false,
    typePanelOpen: false,
    submitting: false,
    submitError: "",
    receiptNumber: result.data.receiptNumber,
    canSubmit: false
  });
});
}
});
===== 文件: pages/feedback/feedback.wxml =====
<view class="page">
  <nav-bar title="用户反馈" fallback-url="/pages/my/my"></nav-bar>
  <view class="content">
    <view wx:if="{{receiptNumber}}" class="success">
      <text class="success__title">反馈已提交</text>
      <view class="success__detail">
        <text class="success__line">受理编号: {{receiptNumber}}</text>
        <text class="success__line">普通咨询 / 投诉: 24 小时内完成核查并反馈结果</text>
      </view>
    </view>
  </view>
</view>

```

```
<text class="success__line">涉及轻生、诈骗、隐私泄露等高风险诉求：2 小时内人工加急处置</text>
</view>
</view>

<text class="label">诉求类型</text>
<view
  class="picker-field {{type ? '': 'picker-field--placeholder'}} {{typePanelOpen ? 'picker-field--open': ''}}"
  bindtap="onToggleTypePanel"
>
  <text class="picker-field__label">{{typeOptions[typeIndex].label}}</text>
  <text class="picker-field__chevron">{{typePanelOpen ? '^': 'v'}}</text>
</view>
<view wx:if="{{typePanelOpen}}" class="type-options">
  <block wx:for="{{typeOptions}}" wx:key="value">
    <view
      wx:if="{{index > 0}}"
      class="type-option {{typeIndex === index ? 'type-option--selected': ''}}"
      data-index="{{index}}"
      bindtap="onSelectType"
    >
      <text class="type-option__label">{{item.label}}</text>
      <text wx:if="{{typeIndex === index}}" class="type-option__check">✓</text>
    </view>
  </block>
</view>

<text class="label">问题详细描述</text>
<view class="description-wrap">
  <textarea class="description" value="{{description}}" maxlength="500" disabled="{{submitting}}" placeholder="请清晰说明发生时间、对话内容和遇到的问题。请勿填写银行卡号、身份证号、密码或验证码等隐私信息。" bindinput="onDescriptionInput"></textarea>
  <text class="count">{{description.length}}/500</text>
</view>

<view class="consent" bindtap="onToggleConsent">
  <view class="checkbox {{consent ? 'checkbox--checked': ''}}">{{consent ? '✓': ''}}</view>
  <text class="consent-copy">我已阅读<text class="policy-link" catchtap="onOpenConsent">《知情同意书》</text>，同意平台仅为受理和处理本次反馈收集、使用上述信息。</text>
</view>

<text wx:if="{{submitError}}" class="submit-error">{{submitError}}</text>
<view class="submit {{canSubmit ? '': 'submit--disabled'}}" bindtap="onSubmit">{{submitting ? '提交中': '提交'}}</view>
</view>

<!-- 知情同意书弹窗：查看不自动勾选，仅点底部按钮视为同意 -->
<view wx:if="{{consentVisible}}" class="consent-mask" catchtouchmove="noop" catchtap="onCloseConsent">
  <view class="consent-dialog" catchtap="noop">
    <view class="consent-dialog__close" catchtap="onCloseConsent">✕</view>
    <text class="consent-dialog__title">知情同意书</text>
    <scroll-view class="consent-dialog__body" scroll-y>
      <text class="consent-section__title">个人信息收集告知</text>
      <text class="consent-section__text">温馨告知：您填写的手机号、联系信息仅用于本次投诉反馈处理，仅留存 6 个月，到期自动脱敏删除；不会用于 AI 模型训练、营销推送，不会向第三方泄露、出售。</text>
      <text class="consent-section__title">AI 拟人服务专项提示</text>
      <text class="consent-section__text">如您反馈 AI 虚拟宠物对话存在违规内容，平台将调取对应时段对话记录开展安全复核，相关对话数据仅用于本次风险核查，核查完毕统一归档留存。</text>
      <text class="consent-section__title">未成年人专项提示</text>
      <text class="consent-section__text">未满 14 周岁用户提交反馈，平台处理后将同步风险提醒至您填写的
```

```
    监护人手机号；涉及心理、极端情绪问题会优先联系监护人介入。</text>
    <text class="consent-section__title">老年人友好提示</text>
    <text class="consent-section__text">老年用户如不会操作表单，可直接拨打客服热线 <text class="consent-hotline" catchtap="onCallHotline">{{hotline}}</text> 人工反馈，无需填写线上表单。</text>
  </scroll-view>
  <view class="consent-dialog__agree" bindtap="onAgreeConsent">我已阅读并同意</view>
</view>
</view>
</view>
===== 文件: pages/profile/profile.js =====
const petStore = require('../../utils/pet-store');
const request = require('../../utils/request');
const auth = require('../../utils/auth');
const { API_BASE_URL } = require('../../config/api');

const PROFILE_KEY = 'eggbaby_profile_v1';
const MBTI_LIST = ['INFP','INFJ','INTJ','INTP','ENFP','ENFJ','ENTJ','ENTP','ISFP','ISFJ','ISTJ','ISTP','ESFP','ESFJ','ESTJ','ESTP'];
const GENDER_LIST = ['男', '女', '保密'];
const GENDER_MAP = { '男': 'MALE', '女': 'FEMALE', '保密': 'OTHER' };
const GENDER_REVERSE = { MALE: '男', FEMALE: '女', OTHER: '保密' };
const CITY_MAX_LENGTH = 10;

const ZODIAC_MAP = {
  aries: '白羊座', taurus: '金牛座', gemini: '双子座',
  cancer: '巨蟹座', leo: '狮子座', virgo: '处女座',
  libra: '天秤座', scorpio: '天蝎座', sagittarius: '射手座',
  capricorn: '摩羯座', aquarius: '水瓶座', pisces: '双鱼座'
};

function translateZodiac(zodiac) {
  if (!zodiac) return '';
  const code = String(zodiac).toLowerCase();
  return ZODIAC_MAP[code] || zodiac;
}

function maskUserId(userId) {
  const s = String(userId || '');
  return s.slice(0, 8);
}

function formatDisplay(profile) {
  const user = petStore.getUser() || {};
  const session = auth.getSession() || {};
  const userId = user.id || session.userId || profile.userId || '';
  return {
    nickname: profile.nickname || user.nickname || '蛋友',
    avatarUrl: profile.avatarUrl || user.avatarUrl || '',
    userId: maskUserId(userId),
    gender: profile.gender ? (GENDER_REVERSE[profile.gender] || '未设置') : '未设置',
    birthday: profile.birthday || '未设置',
    zodiac: translateZodiac(profile.zodiac) || '——',
    city: profile.city || '未设置',
    mbti: profile.mbti || '未设置'
  };
}

Page({
  data: {
    nickname: '蛋友',
```



```
    userId: "",
    gender: '未设置',
    birthday: '未设置',
    zodiac: '——',
    mbti: '未设置',
    mbtiList: MBTI_LIST,
    avatarUrl: ""
  },

  onLoad() {
    this.loadProfile();
  },

  loadProfile() {
    request.get('/wechat/profile')
      .then((profile) => {
        wx.setStorageSync(PROFILE_KEY, profile);
        petStore.syncUserProfile(profile);
        this.setData(formatDisplay(profile));
      })
      .catch(() => {
        const cached = wx.getStorageSync(PROFILE_KEY) || {};
        const user = petStore.getUser() || {};
        const session = auth.getSession() || {};
        this.setData(formatDisplay({ ...cached, userId: user.id || session.userId }));
      });
  },

  refreshProfile() {
    return request.get('/wechat/profile')
      .then((profile) => {
        wx.setStorageSync(PROFILE_KEY, profile);
        petStore.syncUserProfile(profile);
        this.setData(formatDisplay(profile));
        return profile;
      });
  },

  saveProfile(partial) {
    return request.put('/wechat/profile', partial)
      .then(() => {
        const cached = wx.getStorageSync(PROFILE_KEY) || {};
        const next = { ...cached, ...partial };
        wx.setStorageSync(PROFILE_KEY, next);
        petStore.syncUserProfile(next);
        this.setData(formatDisplay(next));
        wx.showToast({ title: '保存成功', icon: 'success' });
      })
      .catch((error) => {
        wx.showToast({ title: (error && error.userMessage) || '保存失败', icon: 'none' });
        throw error;
      });
  },

  onChooseAvatar(e) {
    const tempPath = e.detail.avatarUrl;
    if (!tempPath) return;
    const session = auth.getSession();
    if (!session || !session.token) {
      wx.showToast({ title: '登录状态已失效', icon: 'none' });
    }
  }
}
```

```
    return;
  }
  wx.uploadFile({
    url: `${API_BASE_URL}/wechat/avatar`,
    filePath: tempPath,
    name: 'file',
    header: { Authorization: `Bearer ${session.token}` },
    success: (res) => {
      if (res.statusCode !== 200) {
        wx.showToast({ title: '头像上传失败', icon: 'none' });
        return;
      }
      try {
        const envelope = JSON.parse(res.data);
        if (envelope.code !== 0 || !envelope.data) {
          wx.showToast({ title: envelope.msg || '头像上传失败', icon: 'none' });
          return;
        }
        const avatarUrl = envelope.data;
        this.saveProfile({ avatarUrl });
      } catch (error) {
        wx.showToast({ title: '头像上传失败', icon: 'none' });
      }
    },
    fail: () => wx.showToast({ title: '头像上传失败', icon: 'none' })
  });
},

onEditNickname() {
  wx.showModal({
    title: '修改昵称',
    editable: true,
    placeholderText: '最多 16 个字',
    content: this.data.nickname,
    success: (res) => {
      if (!res.confirm) return;
      const value = String(res.content || '').trim().slice(0, 16);
      if (!value) return;
      this.saveProfile({ nickname: value });
    }
  });
},

onEditGender() {
  wx.showActionSheet({
    itemList: GENDER_LIST,
    success: (result) => {
      const value = GENDER_MAP[GENDER_LIST[result.tapIndex]];
      this.saveProfile({ gender: value });
    }
  });
},

onEditBirthday(e) {
  const value = e.detail.value;
  if (!value) return;
  this.saveProfile({ birthday: value })
    .then(() => this.refreshProfile());
},
```

```

onEditCity() {
  wx.showModal({
    title: '修改常驻城市',
    editable: true,
    placeholderText: `最多 ${CITY_MAX_LENGTH} 个字`,
    content: this.data.city === '未设置' ? '' : this.data.city,
    success: (res) => {
      if (!res.confirm) return;
      const value = String(res.content || '').trim().slice(0, CITY_MAX_LENGTH);
      if (!value) return;
      this.saveProfile({ city: value });
    }
  });
},

onMbtiChange(e) {
  this.saveProfile({ mbti: MBTI_LIST[e.detail.value] });
}
});
===== 文件: pages/profile/profile.wxml =====
<view class="page">
  <nav-bar title="我的信息"></nav-bar>

  <scroll-view scroll-y class="content">

    <view class="avatar-section">
      <button class="avatar-btn" open-type="chooseAvatar" bindchooseavatar="onChooseAvatar">
        <image wx:if="{{avatarUrl}}" class="avatar-image" src="{{avatarUrl}}" mode="aspectFill"></image>
        <egg-avatar wx:else size="{{72}}"></egg-avatar>
      </button>

      <view class="nickname-row" bindtap="onEditNickname">
        <text class="nickname-text">{{nickname}}</text>
        <view class="chevron"></view>
      </view>
      <text class="user-id">ID: {{userId}}</text>
    </view>

    <view class="field-panel">
      <list-row
        label="性别"
        value="{{gender}}"
        bind:rowtap="onEditGender"
      ></list-row>
      <picker mode="date" bindchange="onEditBirthday">
        <list-row label="生日" value="{{birthday}}"></list-row>
      </picker>
      <list-row label="星座" value="{{zodiac}}" show-chevron="{{false}}"></list-row>
      <list-row
        label="常驻城市"
        value="{{city}}"
        bind:rowtap="onEditCity"
      ></list-row>
      <picker mode="selector" range="{{mbtiList}}" bindchange="onMbtiChange">
        <list-row label="MBTI" value="{{mbti}}" show-border="{{false}}"></list-row>
      </picker>
    </view>

  </scroll-view>
</view>

```

===== 文件: pages/nickname/nickname.js =====

```
const petStore = require('../../utils/pet-store');
```

```
Page({
  data: { name: "", count: 0, error: "" },
  onLoad() {
    const pet = petStore.getPet();
    const name = pet ? pet.name : "";
    this.setData({ name, count: Array.from(name).length });
  },
  onInput(e) {
    const name = e.detail.value;
    this.setData({ name, count: Array.from(name).length, error: "" });
  },
  async onSave() {
    const result = await petStore.updateNickname(this.data.name);
    if (!result.ok) return this.setData({ error: result.message });
    wx.showToast({ title: result.alreadyDone ? '昵称已更新' : '它记住了自己的名字', icon: 'none' });
    setTimeout(() => wx.navigateBack(), 700);
  }
});
```

===== 文件: pages/nickname/nickname.wxml =====

```
<view class="page"><nav-bar title="给它起名字"></nav-bar><view class="content"><view class="mini-egg"></view><text class="title">它以后该叫什么? </text><text class="desc">这个名字会一直陪它到破壳以后。</text><view class="input-wrap"><input class="name-input" value="{{name}}" maxlength="10" placeholder="最多 10 个字符" bindinput="onInput" focus/><text class="counter">{{count}} / 10</text></view><text wx:if="{{error}}" class="error">{{error}}</text><view class="spacer"></view><button class="primary" bindtap="onSave">确认这个名字</button></view></view></view>
```

===== 文件: pages/my/my.js =====

```
const petStore = require('../../utils/pet-store');
```

```
const request = require('../../utils/request');
```

```
const inviteApi = require('../../utils/invite-api');
```

```
const SHARE_TITLE = '一起来养蛋宝宝吧';
```

```
const SHARE_HOME_PATH = '/pages/home/home?v=1&source=home_share';
```

```
Page({
  data: { userName: '蛋友', avatarUrl: "", eggCount: 0, shareReady: false },

  onShow() {
    // 自定义悬浮 tabBar: 标记当前 tab; 在「我的」页不渲染任何圆形按钮
    if (typeof this.getTabBar === 'function') {
      const tabBar = this.getTabBar();
      if (tabBar) tabBar.setData({ selected: 1, hidden: true });
    }
    this.loadUserProfile();
    this.loadPetStatus();
    this.prepareShare();
  },

  prepareShare() {
    const requestId = (this._shareRequestId || 0) + 1;
    this._shareRequestId = requestId;
    this.setData({ shareReady: false });
    if (typeof wx !== 'undefined' && wx.hideShareMenu) {
      wx.hideShareMenu({ menus: ['shareAppMessage', 'shareTimeline'] });
    }
    this.loadShareInviteCode(requestId);
  },
});
```

```

async loadShareInviteCode(requestId) {
  const currentRequestId = requestId === undefined
    ? (this._shareRequestId = (this._shareRequestId || 0) + 1)
    : requestId;
  this._shareInviteCode = null;
  try {
    const inviteCode = await inviteApi.getMine();
    if (this._shareRequestId !== currentRequestId) return;
    if (this.isUsableShareInvite(inviteCode)) this._shareInviteCode = inviteCode.code;
  } catch (error) {
    if (this._shareRequestId !== currentRequestId) return;
    this._shareInviteCode = null;
  } finally {
    if (this._shareRequestId !== currentRequestId) return;
    this.setData({ shareReady: true });
    if (typeof wx !== 'undefined' && wx.showShareMenu) {
      wx.showShareMenu({ menus: ['shareAppMessage', 'shareTimeline'] });
    }
  }
},

isUsableShareInvite(inviteCode) {
  return !!(inviteCode && inviteCode.code && inviteCode.remaining > 0 && inviteCode.status === 1);
},

loadUserProfile() {
  request.get('/wechat/profile')
    .then((profile) => {
      petStore.syncUserProfile(profile);
      this.applyUserData();
    })
    .catch(() => this.applyUserData());
},

applyUserData() {
  const user = petStore.getUser() || {};
  this.setData({ userName: user.nickname || '蛋友', avatarUrl: user.avatarUrl || '' });
},

loadPetStatus() {
  const pet = petStore.getPet();
  this.setData({ eggCount: pet ? 1 : 0 });
},

onTapUserCard() { wx.navigateTo({ url: '/pages/profile/profile' }); },
onNavAlbum() { wx.navigateTo({ url: '/pages/album/album' }); },
onNavCodes() { wx.navigateTo({ url: '/pages/invite-codes/invite-codes' }); },
onNavSettings() { wx.navigateTo({ url: '/pages/settings/settings' }); },
onNavChatSettings() { wx.navigateTo({ url: '/pages/chat-settings/chat-settings' }); },
onNavAccount() { wx.navigateTo({ url: '/pages/account/account' }); },
onNavHelp() { wx.navigateTo({ url: '/pages/help/help' }); },

onShareAppMessage() {
  if (!this.data.shareReady) return false;
  const inviteCode = this._shareInviteCode;
  return {
    title: SHARE_TITLE,
    path: inviteCode ? `${SHARE_HOME_PATH}&inviteCode=${encodeURIComponent(inviteCode)}` : SHARE_HOME_PATH
  };
};

```

```

},

onShareTimeline() {
  if (!this.data.shareReady) return false;
  return { title: SHARE_TITLE };
}
});
===== 文件: pages/my/my.wxml =====
<view class="page">
  <nav-bar title="我的" fallback-url="/pages/home/home"></nav-bar>

  <view class="content">

    <view class="user-card" hover-class="user-card--pressed" bindtap="onTapUserCard">
      <image wx:if="{{avatarUrl}}" class="user-avatar" src="{{avatarUrl}}" mode="aspectFill"></image>
      <egg-avatar wx:else size="{{56}}" gradient-from="#9DB65B" gradient-to="#EDE78E"></egg-avatar>
      <view class="user-info">
        <text class="user-name">{{userName}}</text>
        <text class="user-sub">{{eggCount}} 只蛋宝宝</text>
      </view>
      <view class="chevron"></view>
    </view>

    <view class="menu-panel">
      <list-row label="我的卡册" bind:rowtap="onNavAlbum"></list-row>
      <list-row label="我的激活码" bind:rowtap="onNavCodes"></list-row>
      <list-row label="对话记录" bind:rowtap="onNavChatSettings"></list-row>
      <list-row label="系统设置" bind:rowtap="onNavSettings"></list-row>
      <!-- 账号，后期打开 -->
      <!-- <list-row label="账号" bind:rowtap="onNavAccount"></list-row> -->
      <list-row label="帮助中心" show-border="{{false}}" bind:rowtap="onNavHelp"></list-row>
    </view>

  </view>
</view>
===== 文件: pages/chat-settings/chat-settings.js =====
const request = require('../../utils/request');

const EMAIL_PATTERN = /^^[^s@]+@[^[^s@]+\.[^s@]+$/;
const EXPORT_EMAIL_KEY = 'export_email';

Page({
  data: {
    email: '',
    emailFocus: false,
    error: '',
    submitting: false,
    deleting: false
  },

  onLoad() {
    const savedEmail = wx.getStorageSync(EXPORT_EMAIL_KEY);
    if (savedEmail) {
      this.setData({ email: savedEmail });
    }
  },

  onEmailInput(e) {
    this.setData({ email: e.detail.value.trim(), error: '' });
  },

```

```
onEmailFocus() {
  this.setData({ emailFocus: true });
},

onEmailBlur() {
  this.setData({ emailFocus: false });
},

onExport() {
  const { email, submitting } = this.data;
  if (submitting) return;

  if (!email) {
    this.setData({ error: '请输入接收邮箱' });
    return;
  }
  if (!EMAIL_PATTERN.test(email)) {
    this.setData({ error: '邮箱格式不正确' });
    return;
  }

  this.setData({ submitting: true });
  request.post('/wechat/chat-history/export', { email })
    .then(() => {
      wx.setStorageSync(EXPORT_EMAIL_KEY, email);
      this.setData({ email: '' });
      wx.showModal({
        title: '导出成功',
        content: `聊天记录已开始导出，稍后将发送至 ${email}，请注意查收。`,
        showCancel: false,
        confirmText: '知道了',
        success: () => {
          wx.navigateBack({ delta: 1 });
        }
      });
    })
    .catch((err) => {
      const message = (err && err.userMessage) || '导出失败，请稍后重试';
      wx.showToast({ title: message, icon: 'none' });
    })
    .finally(() => {
      this.setData({ submitting: false });
    });
},

onDelete() {
  if (this.data.deleting) return;

  wx.showModal({
    title: '删除聊天记录',
    content: '删除后将清空你的全部对话记录且无法恢复，确定删除吗？',
    confirmText: '仍要删除',
    confirmColor: '#C33F36',
    cancelText: '取消',
    success: (res) => {
      if (res.confirm) {
        this.confirmDelete();
      }
    }
  })
}
```

```

});
},

confirmDelete() {
  this.setData({ deleting: true });
  request.post('/wechat/chat-history/delete')
    .then(() => {
      wx.showModal({
        title: '删除成功',
        content: '你的全部聊天记录已删除。',
        showCancel: false,
        confirmText: '知道了',
        success: () => {
          wx.navigateBack({ delta: 1 });
        }
      });
    })
    .catch((err) => {
      const message = (err && err.userMessage) || '删除失败，请稍后重试';
      wx.showToast({ title: message, icon: 'none' });
    })
    .finally(() => {
      this.setData({ deleting: false });
    });
}
});
===== 文件: pages/chat-settings/chat-settings.wxml =====
<view class="page">
  <nav-bar title="对话记录"></nav-bar>

  <view class="content">

    <view class="section">
      <text class="section-title">导出聊天记录</text>
      <text class="section-desc">将你的全部对话整理成文件，发送到指定邮箱。</text>

      <view class="input-wrap {{emailFocus ? 'input-wrap--focus' : ''}} {{error ? 'input-wrap--error' : ''}}">
        <input
          class="email-input"
          type="text"
          value="{{email}}"
          placeholder="请输入接收邮箱"
          placeholder-class="placeholder"
          bindinput="onEmailInput"
          bindfocus="onEmailFocus"
          bindblur="onEmailBlur"
        />
      </view>
      <text wx:if="{{error}}" class="error">{{error}}</text>

      <button
        class="primary"
        hover-class="primary--pressed"
        loading="{{submitting}}"
        disabled="{{submitting}}"
        bindtap="onExport"
      >{{submitting ? '提交中...' : '导出并发送到邮箱'}}</button>
    </view>
  </view>

```



```

<view class="section">
  <text class="section-title">删除聊天记录</text>
  <text class="section-desc">删除你的全部对话记录，删除后无法恢复。建议先导出备份。</text>

  <button
    class="danger"
    hover-class="danger--pressed"
    loading="{{deleting}}"
    disabled="{{{deleting}}}"
    bindtap="onDelete"
  >{{deleting ? '删除中...' : '删除全部聊天记录'}}</button>
</view>

</view>
</view>
===== 文件: pages/hatch-guide/hatch-guide.js =====
const petStore = require('../../utils/pet-store');
const petApi = require('../../utils/pet-api');

Page({
  data: { pet: null, tasks: [] },

  async onShow() {
    const pet = petStore.getPet();
    if (!pet) {
      wx.switchTab({ url: '/pages/home/home' });
      return;
    }
    // 从后端拉修炼动作列表，缓存到 pet._hatchActions 供 getHatchActionState 派生
    if (pet.hatchStatus !== 'HATCHED') {
      try {
        const actions = await petApi.listHatchActions(pet.id);
        pet._hatchActions = Array.isArray(actions) ? actions : [];
        petStore.savePet(pet);
      } catch (error) {
        // 拉取失败则沿用旧缓存或 tasks 默认值，不阻塞渲染
      }
    }
    const state = petStore.getHatchActionState(pet);
    this.setData({
      pet,
      tasks: [
        { key: 'nickname', title: '给蛋宝宝起昵称', desc: '让它知道自己是谁', done: state.nicknameDone,
route: '/pages/nickname/nickname' },
        { key: 'cuddle', title: '贴贴蛋宝宝', desc: '回首页长按蛋壳 3 秒', done: state.cuddleDone, route
: 'home', hint: '长按蛋壳 3 秒完成贴贴' },
        { key: 'wish', title: '今日许愿', desc: '告诉它你期待怎样的陪伴', done: state.wishDone, route: '
/pages/wish/wish' },
        { key: 'lesson', title: '蛋前教育', desc: '今天想教它一件什么事', done: state.lessonDone, route:
'/pages/lesson/lesson' },
        { key: 'doodle', title: '彩蛋涂鸦', desc: '回首页点「画画」画下记号', done: state.doodleDone, ro
ute: 'home', hint: '在首页点「画画」开始涂鸦' }
      ]
    });
  },

  onTask(e) {
    const route = e.currentTarget.dataset.route;
    if (route === 'home') {
      wx.switchTab({ url: '/pages/home/home' });
    }
  }
});

```

```

const hint = e.currentTarget.dataset.hint;
if (hint) setTimeout(() => wx.showToast({ title: hint, icon: 'none' }), 300);
return;
}
wx.navigateTo({ url: route });
});
===== 文件: pages/hatch-guide/hatch-guide.wxml =====
<view class="page">
  <nav-bar title="孵化修炼手册"></nav-bar>
  <scroll-view class="content" scroll-y>
    <view class="progress-card">
      <view class="progress-copy"><text class="progress-kicker">HATCHING</text><text class="progress-number">{{pet.progress}}%</text></view>
      <view class="progress-track"><view class="progress-fill" style="width: {{pet.progress}}%;"></view>
    </view>
    <text class="progress-note">每天做一两件就好，不需要一次全部完成。</text>
  </view>

  <text class="section-title">今天可以做</text>
  <view class="task-list">
    <view class="task {{item.done ? 'task--done': ''}} wx:for="{{tasks}}" wx:key="key" data-route="{{item.route}}" data-hint="{{item.hint}}" bindtap="onTask" hover-class="pressable--pressed">
      <view class="task-status"><text>{{item.done ? '✓': ''}}</text></view>
      <view class="task-copy"><text class="task-title">{{item.title}}</text><text class="task-desc">{{item.desc}}</text></view>
      <view class="task-side"><text class="task-arrow">></text></view>
    </view>
  </view>
  <!-- <view class="quiet-note">孵化进度影响反馈、性格和收藏卡内容，不会改变物理装置的预设打开时间。</view> -->
</scroll-view>
</view>
===== 文件: pages/lesson/lesson.js =====
const petStore = require('../../utils/pet-store');
Page({
  data: { selected: '', options: [
    { icon: '🐡', value: '去深海潜水' },
    { icon: '☁️', value: '做个云朵棉花糖SPA' },
    { icon: '🌙', value: '月光牛奶浴' },
    { icon: '🎠', value: '坐星光旋转木马' },
    { icon: '🎧', value: '听树洞收音机' },
    { icon: '🍞', value: '在面包房打盹' }
  ] },
  onSelect(e) { this.setData({ selected: e.currentTarget.dataset.value }); },
  async onSubmit() {
    if (!this.data.selected) return wx.showToast({ title: '先选一堂课吧', icon: 'none' });
    const result = await petStore.completeLesson(this.data.selected);
    if (!result.ok) return wx.showToast({ title: result.message, icon: 'none' });
    wx.showToast({ title: result.alreadyDone ? '今天已经上过课啦': '它认真听完了', icon: 'none' });
    setTimeout(() => wx.navigateBack(), 700);
  }
});
===== 文件: pages/lesson/lesson.wxml =====
<view class="page"><nav-bar title="蛋前教育"></nav-bar><view class="content"><text class="kicker">LITTLE
  CLASS</text><text class="title">今天想教蛋宝宝什么? </text><text class="desc">你的选择会轻轻影响它
  破壳后的性格。</text><view class="options"><view wx:for="{{options}}" wx:key="value" class="option {{
  selected === item.value ? 'option--selected': ''}} data-value="{{item.value}}" bindtap="onSelect"
  ><view class="icon">{{item.icon}}</view><text>{{item.value}}</text><text class="arrow">{{selected ===
  item.value ? '✓': ''}}</text></view></view><view class="spacer"></view><button class="primary" {{

```

```
!selected ? 'primary--disabled' : ''}" bindtap="onSubmit">开始今天的小课</button></view></view>
```

```
===== 文件: pages/account/account.js =====
```

```
const petStore = require('../../utils/pet-store');
```

```
Page({
  onLogout() {
    wx.showModal({
      title: '退出登录',
      content: '退出后将清除本机账号及蛋宝宝体验数据。',
      confirmColor: '#D9463C',
      success: (res) => {
        if (!res.confirm) return;
        getApp().clearLoginState();
        petStore.clearAccountData();
        wx.reLaunch({ url: '/pages/welcome/welcome' });
      }
    });
  },

  onDeregister() { wx.navigateTo({ url: '/pages/deregister/deregister' }); }
});
```

```
===== 文件: pages/account/account.wxml =====
```

```
<view class="page">
  <nav-bar title="账号"></nav-bar>
```

```
<view class="content">
```

```
  <view class="panel">
    <list-row
      label="退出登录"
      show-chevron="{{false}}"
      show-border="{{false}}"
      bind:rowtap="onLogout"
    ></list-row>
  </view>
```

```
<view class="spacer"></view>
```

```
<view class="panel danger-panel">
  <list-row
    label="注销账号"
    label-color="#D9463C"
    show-chevron="{{false}}"
    show-border="{{false}}"
    bind:rowtap="onDeregister"
  ></list-row>
</view>
```

```
</view>
```

```
</view>
```

```
===== 文件: pages/invite-codes/invite-codes.js =====
```

```
const inviteApi = require('../../utils/invite-api');
```

```
const MESSAGES = {
  load: '加载失败，请下拉重试',
  empty: '暂无可用激活码'
};
```

```
Page({
  data: {
```

```

inviteCode: null,
exhausted: false,
loading: true,
error: "
},

onShow() {
  this.loadInviteCode();
},

async loadInviteCode() {
  this.setData({ loading: true, error: " });
  try {
    const code = await inviteApi.getMine();
    const exhausted = this.isExhausted(code);
    this.setData({ inviteCode: code, exhausted, loading: false, error: " });
  } catch (error) {
    this.setData({
      inviteCode: null,
      exhausted: true,
      loading: false,
      error: (error && error.userMessage) || MESSAGES.load
    });
  }
},

```

```

isExhausted(code) {
  if (!code) return true;
  return (code.remaining || 0) <= 0 || code.status !== 1;
},

```

```

onRetry() {
  this.loadInviteCode();
},

```

```

onCopy() {
  const { inviteCode, exhausted, loading } = this.data;
  if (loading || !inviteCode || exhausted) return;
  wx.setClipboardData({ data: inviteCode.code });
}
});

```

===== 文件: pages/invite-codes/invite-codes.wxml =====

```

<view class="page">
  <nav-bar title="我的激活码"></nav-bar>
  <scroll-view class="content" scroll-y>
    <view wx:if="{{loading}}" class="state state--loading">
      <text class="state-text">加载中...</text>
    </view>

    <view wx:elif="{{error}}" class="state state--error">
      <text class="state-text">{{error}}</text>
      <button class="retry" bindtap="onRetry">重试</button>
    </view>

    <block wx:elif="{{inviteCode}}">
      <view class="intro">
        <text class="title">把一颗蛋送给朋友</text>
        <text class="desc">你有一个专属激活码，可邀请朋友各领养一只蛋宝宝。每位朋友只能使用一次，使用后扣减一次剩余次数。</text>
      </view>
    </block>
  </scroll-view>
</view>

```

```

<view class="code-card" {{exhausted ? 'code-card--exhausted' : ''}}">
  <view class="code-head">
    <text class="code-label">我的激活码</text>
    <text class="usage">已用 {{inviteCode.usedCount}} / {{inviteCode.quota}} · 剩余 {{inviteCode.r
emaining}}</text>
  </view>
  <text class="code">{{inviteCode.code}}</text>
  <view class="code-foot">
    <text class="state-label">{{exhausted ? '已用完' : '可邀请'}}</text>
    <button class="copy" disabled="{{exhausted}}" bindtap="onCopy">复制激活码</button>
  </view>
</view>
</block>

<view wx:else class="empty">暂无可用激活码</view>

<view class="note">个人激活码不提供积分、排行榜或现金奖励。只有朋友成功绑定蛋宝宝后，才会扣减一次剩
余次数。</view>
</scroll-view>
</view>
===== 文件: pages/album/album.js =====
const petStore = require('../../utils/pet-store');
Page({
  data: { cards: [] },
  onShow() {
    const pet = petStore.getPet();
    const cards = (pet && pet.collectionCards) ? pet.collectionCards.map((card, index) => ({
      ...card,
      petType: pet.prototype || '玉兔',
      name: pet.name || pet.prototype || '蛋宝宝',
      index
    })): [];
    this.setData({ cards });
  },
  onOpen(e) {
    const index = e.currentTarget.dataset.index || 0;
    wx.navigateTo({ url: '/pages/collection-card/collection-card?index=${index}' });
  }
});
===== 文件: pages/album/album.wxml =====
<view class="page">
  <nav-bar title="我的卡册"></nav-bar>
  <view class="content">
    <view wx:if="{{cards.length > 0}}" class="card-list">
      <view wx:for="{{cards}}" wx:key="id" class="album-card" bindtap="onOpen" data-index="{{item.index}}
">
        <image wx:if="{{item.imageUrl}}" class="album-pet-avatar" src="{{item.imageUrl}}" mode="aspectFi
ll"></image>
        <view wx:else class="baby-mark">{{item.petType === '锦鲤' ? '锦' : '兔'}}</view>
        <view class="copy">
          <text class="name">{{item.name}}</text>
          <text class="meta">{{item.brief || ''}}</text>
          <text class="source-tag">{{item.source === 'HATCH' ? '破壳首卡' : item.source}}</text>
        </view>
        <text class="arrow">></text>
      </view>
    </view>
    <view wx:else class="empty">
      <view class="empty-card"></view>
      <text class="empty-title">卡册还是空的</text>
    </view>
  </view>
</view>

```

```

      <text class="empty-desc">到达预设破壳日后，收藏卡会自动收入这里。</text>
    </view>
  </view>
</view>
===== 文件: pages/deregister/deregister.js =====
const DEREGISTER_KEY = 'eggbaby_deregister_request_v1';

Page({
  data: { pending: false, endDate: '' },

  onShow() {
    const request = wx.getStorageSync(DEREGISTER_KEY);
    if (!request) return this.setData({ pending: false, endDate: '' });
    const end = new Date(request.endTime);
    this.setData({
      pending: true,
      endDate: `${end.getFullYear()}年${end.getMonth() + 1}月${end.getDate()}日`
    });
  },

  onConfirmDeregister() {
    if (this.data.pending) {
      wx.showToast({ title: '注销申请已提交，可取消注销', icon: 'none' });
      return;
    }
    wx.showModal({
      title: '再次确认',
      content: '提交后进入 15 天冷静期，数据不会立即删除。确定继续吗?',
      confirmColor: '#D9463C',
      success: (res) => {
        if (!res.confirm) return;
        const submittedAt = Date.now();
        wx.setStorageSync(DEREGISTER_KEY, { submittedAt, endTime: submittedAt + 15 * 24 * 60 * 60 * 1000 });
      },
      this.onShow();
      wx.showToast({ title: '注销申请已提交', icon: 'success' });
    });
  },

  onCancelDeregister() {
    wx.showModal({
      title: '取消注销',
      content: '取消后账号会恢复正常，可以继续使用蛋宝宝、卡册和对话。',
      success: (res) => {
        if (!res.confirm) return;
        wx.removeStorageSync(DEREGISTER_KEY);
        this.onShow();
        wx.showToast({ title: '已取消注销', icon: 'success' });
      }
    });
  },

  onBackAccount() { wx.navigateBack(); },
  onCancel() { wx.navigateBack(); }
});
===== 文件: pages/deregister/deregister.wxml =====
<view class="page">
  <nav-bar title="注销账号"></nav-bar>

```

```
<scroll-view scroll-y class="content">

<view wx:if="{{pending}}" class="pending-block">
  <view class="pending-icon">✓</view>
  <text class="pending-title">注销申请已提交</text>
  <text class="pending-desc">账号正处于 15 天冷静期，预计 {{endDate}} 后处理。数据尚未删除，冷静期结束前可随时取消。</text>
  <text class="pending-note">请勿重复提交注销申请。</text>
</view>

<view wx:else class="warn-block">
  <view class="warn-badge">
    <text class="warn-mark">!</text>
  </view>
  <text class="warn-title">注销后以下数据将被永久删除</text>
</view>

<card wx:if="{{!pending}}" tint="base" ext-class="deletion-card">
  <view class="deletion-list">
    <view class="deletion-row">
      <view class="x-badge"><text class="x-mark">×</text></view>
      <text class="deletion-text">账号信息（昵称、头像、生日、城市等）</text>
    </view>
    <view class="deletion-row">
      <view class="x-badge"><text class="x-mark">×</text></view>
      <text class="deletion-text">所有蛋宝宝的对话记录</text>
    </view>
    <view class="deletion-row">
      <view class="x-badge"><text class="x-mark">×</text></view>
      <text class="deletion-text">蛋宝宝设备绑定关系</text>
    </view>
    <view class="deletion-row">
      <view class="x-badge"><text class="x-mark">×</text></view>
      <text class="deletion-text">成就与成长记录</text>
    </view>
  </view>
</card>

<card wx:if="{{!pending}}" tint="base" ext-class="hardware-card">
  <text class="card-title">关于实体蛋宝宝</text>
  <text class="card-body">注销后设备将解除绑定，回到未激活状态。可用其他账号重新激活，但那将是一只全新的蛋宝宝，原有的对话与记忆不会恢复。</text>
</card>

<card wx:if="{{!pending}}" tint="tint-green" ext-class="cooldown-card">
  <text class="card-title card-title--green">注意</text>
  <view class="cooldown-list">
    <text class="cooldown-item">· 提交申请后进入 <text class="cooldown-strong">15 天冷静期</text></text>
    <text class="cooldown-item">· 冷静期内重新登录可随时撤销注销</text>
    <text class="cooldown-item">· 15 天后数据将被永久删除，无法恢复</text>
  </view>
</card>

</scroll-view>

<view class="button-bar">
  <app-button wx:if="{{pending}}" text="取消注销" bind:tap="onCancelDeregister"></app-button>
  <app-button wx:if="{{pending}}" text="返回账号页" variant="secondary" bind:tap="onBackAccount"></app-button>
</view>
```

```

<app-button wx:if="{{!pending}}" text="我已了解，继续注销" variant="danger" bind:tap="onConfirmDeregister"></app-button>
<app-button wx:if="{{!pending}}" text="我再想想" variant="secondary" bind:tap="onCancel"></app-button>
</view>

```

```

</view>

```

```

===== 文件: pages/help/help.js =====

```

```

// 客服邮箱：页面唯一的联系方式入口，首屏与底部支持条共用

```

```

const SUPPORT_EMAIL = 'hello@eggbabe.com';

```

```

Page({

```

```

  data: {

```

```

    supportEmail: SUPPORT_EMAIL,

```

```

    cats: { device: true, account: true, chat: true, other: true },

```

```

    // 搜索关键词与命中结果：为空串时展示分类列表

```

```

    searchQuery: '',

```

```

    searchResults: [],

```

```

    // 现实求助热线：静态内容（心理援助 12356 / 报警 110 / 急救 120）

```

```

    support: { psychologicalHotline: '12356', police: '110', medicalEmergency: '120' },

```

```

    deviceFaqs: [

```

```

      {

```

```

        q: '如何激活我的蛋宝宝？',

```

```

        a: '打开小程序后点击「添加蛋宝宝」，手动输入随实体蛋或朋友提供的有效激活码。服务端确认后才会完成绑定。',

```

```

        open: false

```

```

      },

```

```

      {

```

```

        q: '互动会让实体装置提前打开吗？',

```

```

        a: '不会。实体装置只按服务端预设时间打开；触摸、说话和蛋壳创作只提供当下回应，不改变时间、款式或收藏卡内容。',

```

```

        open: false

```

```

      },

```

```

      {

```

```

        q: '为什么不能添加第二只蛋宝宝？',

```

```

        a: '当前 MVP 是单蛋版本，每个账号最多绑定 1 只。已绑定账号再次输入激活码时不会消耗新激活码。',

```

```

        open: false

```

```

      },

```

```

      {

```

```

        q: '装置没有按时打开怎么办？',

```

```

        a: '请保留装置照片、激活码信息和问题发生时间，并发送邮件至 ${SUPPORT_EMAIL}。邮件中请说明提前打开、未打开、机械故障或激活码与原型不符等具体情况。',

```

```

        open: false

```

```

      }

```

```

    ],

```

```

    accountFaqs: [

```

```

      {

```

```

        q: '如何修改昵称 / 头像？',

```

```

        a: '进入「我的」→「个人信息」。点击头像区域可更换头像（微信会弹出头像选择面板，第一项即你的微信头像）；点击昵称旁的文字即可修改昵称，最多 16 个字。',

```

```

        open: true

```

```

      }

```

```

    ],

```

```

    chatFaqs: [

```

```

      {

```

```

        q: '为什么现在不能和蛋宝宝对话？',

```

```

        a: '文字对话会在我破壳后开放。孵化期可以轻触或长按蛋壳，从首页自由陪伴入口选择想做的事，也可以直

```



```

    接跟我说说话。',
    open: false
  },
  {
    q: '对话记录会保存多久? ',
    a: '对话只按连续性与安全所必需的范围保存，具体保存期限和删除方式以备案域名上的正式隐私政策为准。',
    open: false
  },
  {
    q: '如何导出或删除对话记录? ',
    a: '进入「我的」→「对话记录」。你可以填写邮箱申请导出，也可以删除当前账号下的全部对话记录。删除后无法恢复，建议先导出备份。',
    open: false
  },
  {
    q: '什么是破壳? ',
    a: '破壳是我结束孵化、正式苏醒的时刻——从这一刻起，我会开口说话，和你开始真正的陪伴与对话。',
    open: false
  }
],

otherFaqs: [
  {
    q: '如何联系邮件支持? ',
    a: '发送邮件至 ${SUPPORT_EMAIL}，并在邮件中说明蛋宝宝 ID、问题发生时间和具体情况。点击本页底部邮箱可以复制地址。',
    open: false
  },
  {
    q: '隐私数据如何处理? ',
    a: '你的数据仅用于提供蛋宝宝服务，具体收集与使用范围见「系统设置」→「隐私政策」。对话记录可在「我的」→「对话记录」中申请导出或删除，其他个人信息权利申请可发送邮件至 ${SUPPORT_EMAIL}。',
    open: false
  }
]
},

onToggleCat(e) {
  const cat = e.currentTarget.dataset.cat;
  this.setData({ [ `cats.${cat}` ]: !this.data.cats[cat] });
},

onToggleFaq(e) {
  const { group, index } = e.currentTarget.dataset;
  this.setData({ [ `${group}.${index}.open` ]: !this.data[group][index].open });
},

// 搜索：跨四组 FAQ 做问题+答案的子串匹配；11 条短文本，量小无需防抖
onSearchInput(e) {
  const searchQuery = String(e.detail.value || '').trim();
  if (!searchQuery) {
    this.setData({ searchQuery: '', searchResults: [] });
    return;
  }
  const groups = ['deviceFaqs', 'accountFaqs', 'chatFaqs', 'otherFaqs'];
  const searchResults = groups
    .flatMap(group => this.data[group] || [])
    .filter(item => `${item.q} ${item.a}`.includes(searchQuery));
  this.setData({ searchQuery, searchResults });
}

```

```

},

// 复制客服邮箱；失败时弹窗展示邮箱地址兜底
onCopySupportEmail() {
  wx.setClipboardData({
    data: SUPPORT_EMAIL,
    fail: () => wx.showModal({
      title: '邮件支持',
      content: SUPPORT_EMAIL,
      showCancel: false
    })
  });
},

// 跳转用户反馈页
onFeedback() {
  wx.navigateTo({ url: '/pages/feedback/feedback' });
}
});
===== 文件: pages/help/help.wxml =====
<view class="page">
  <nav-bar title="帮助中心"></nav-bar>

  <!-- 顶部搜索框：本地过滤 FAQ 关键词 -->
  <view class="search-bar">
    <view class="search-input">
      <view class="search-icon"></view>
      <input class="search-field" value="{{searchQuery}}" placeholder="搜索问题关键词" placeholder-class="search-ph" bindinput="onSearchInput" />
    </view>
  </view>

  <scroll-view scroll-y class="faq-scroll">

    <!-- 首屏：用户反馈入口 + 客服邮箱 + 现实求助信息（静态内容，来自 UI 设计稿）；搜索时隐藏 -->
    <view wx:if="{{!searchQuery}}" class="help-first-screen">
      <text class="help-heading">遇到问题</text>
      <view class="help-action" aria-role="button" bindtap="onFeedback"><text>用户反馈</text><text></text></view>
      <view class="support-config" aria-role="button" aria-label="复制客服邮箱 {{supportEmail}}" bindtap="onCopySupportEmail">
        <view>
          <text class="support-config__label">客服邮箱</text>
          <text class="support-config__value">{{supportEmail}}</text>
        </view>
        <text class="support-config__copy">复制</text>
      </view>
      <text class="help-heading help-heading--reality">需要现实帮助</text>
      <view class="reality-card">
        <text>心理援助热线： {{support.psychologicalHotline}}</text>
        <text>紧急危险： {{support.police}} / {{support.medicalEmergency}}</text>
      </view>
    </view>

    <!-- 搜索结果：命中即定位，搜索时替换分类列表 -->
    <view wx:if="{{searchQuery}}" class="search-results">
      <view wx:if="{{searchResults.length}}">
        <view wx:for="{{searchResults}}" wx:key="q" class="search-result">
          <text class="search-result-q">{{item.q}}</text>
          <text class="search-result-a">{{item.a}}</text>
        </view>
      </view>
    </view>
  </scroll-view>
</view>

```

```
    </view>
  </view>
  <view wx:else class="search-empty">没有找到相关问题，试试其他关键词。</view>
</view>
```

```
<collapse-item wx:if="{{!searchQuery}}"
  title="设备问题"
  variant="category"
  expanded="{{cats.device}}"
  bind:toggle="onToggleCat"
  data-cat="device"
>
  <view class="qa-group">
    <collapse-item
      wx:for="{{deviceFaqs}}"
      wx:key="q"
      title="{{item.q}}"
      variant="question"
      expanded="{{item.open}}"
      show-border="{{index !== deviceFaqs.length - 1}}"
      bind:toggle="onToggleFaq"
      data-group="deviceFaqs"
      data-index="{{index}}"
    >
      <text>{{item.a}}</text>
    </collapse-item>
  </view>
</collapse-item>
```

```
<collapse-item wx:if="{{!searchQuery}}"
  title="账号"
  variant="category"
  expanded="{{cats.account}}"
  bind:toggle="onToggleCat"
  data-cat="account"
>
  <view class="qa-group">
    <collapse-item
      wx:for="{{accountFaqs}}"
      wx:key="q"
      title="{{item.q}}"
      variant="question"
      expanded="{{item.open}}"
      show-border="{{index !== accountFaqs.length - 1}}"
      bind:toggle="onToggleFaq"
      data-group="accountFaqs"
      data-index="{{index}}"
    >
      <text>{{item.a}}</text>
    </collapse-item>
  </view>
</collapse-item>
```

```
<collapse-item wx:if="{{!searchQuery}}"
  title="对话功能"
  variant="category"
  expanded="{{cats.chat}}"
  bind:toggle="onToggleCat"
  data-cat="chat"
>
```

```
<view class="qa-group">
  <collapse-item
    wx:for="{{chatFaqs}}"
    wx:key="q"
    title="{{item.q}}"
    variant="question"
    expanded="{{item.open}}"
    show-border="{{index !== chatFaqs.length - 1}}"
    bind:toggle="onToggleFaq"
    data-group="chatFaqs"
    data-index="{{index}}"
  >
    <text>{{item.a}}</text>
  </collapse-item>
</view>
</collapse-item>

<collapse-item wx:if="{{!searchQuery}}"
  title="其他"
  variant="category"
  expanded="{{cats.other}}"
  show-border="{{false}}"
  bind:toggle="onToggleCat"
  data-cat="other"
>
  <view class="qa-group">
    <collapse-item
      wx:for="{{otherFaqs}}"
      wx:key="q"
      title="{{item.q}}"
      variant="question"
      expanded="{{item.open}}"
      show-border="{{index !== otherFaqs.length - 1}}"
      bind:toggle="onToggleFaq"
      data-group="otherFaqs"
      data-index="{{index}}"
    >
      <text>{{item.a}}</text>
    </collapse-item>
  </view>
</collapse-item>

<view class="bottom-space"></view>

</scroll-view>

<!-- 底部邮件支持条: 点击复制客服邮箱 -->
<view class="support-bar">
  <text class="support-hint">没找到答案? </text>
  <view class="support-link" aria-role="button" aria-label="复制邮件支持邮箱 {{supportEmail}}" bindtap
    =onCopySupportEmail">
    <view class="support-copy">
      <text class="support-label">邮件支持</text>
      <text class="support-email">{{supportEmail}}</text>
    </view>
    <text class="support-action">复制</text>
  </view>
</view>

</view>
```